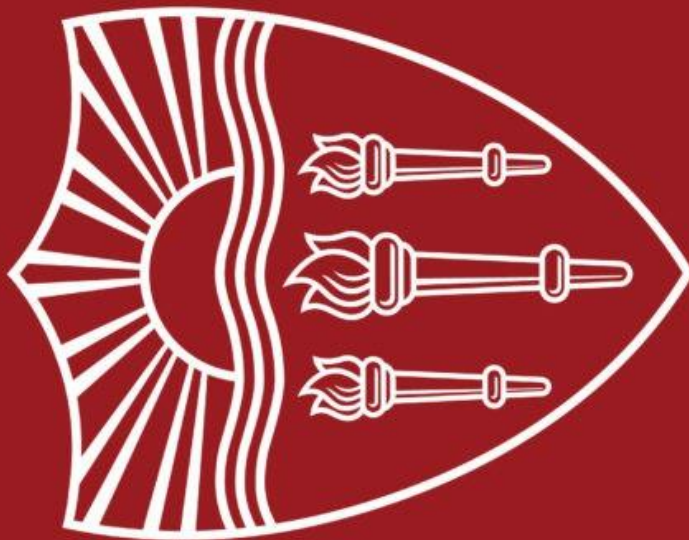




Lecture 8: Feed-forward Neural Nets

Instructor: Xiang Ren
USC CSCI 544 Applied NLP
Fall 2026

Announcements + Logistics

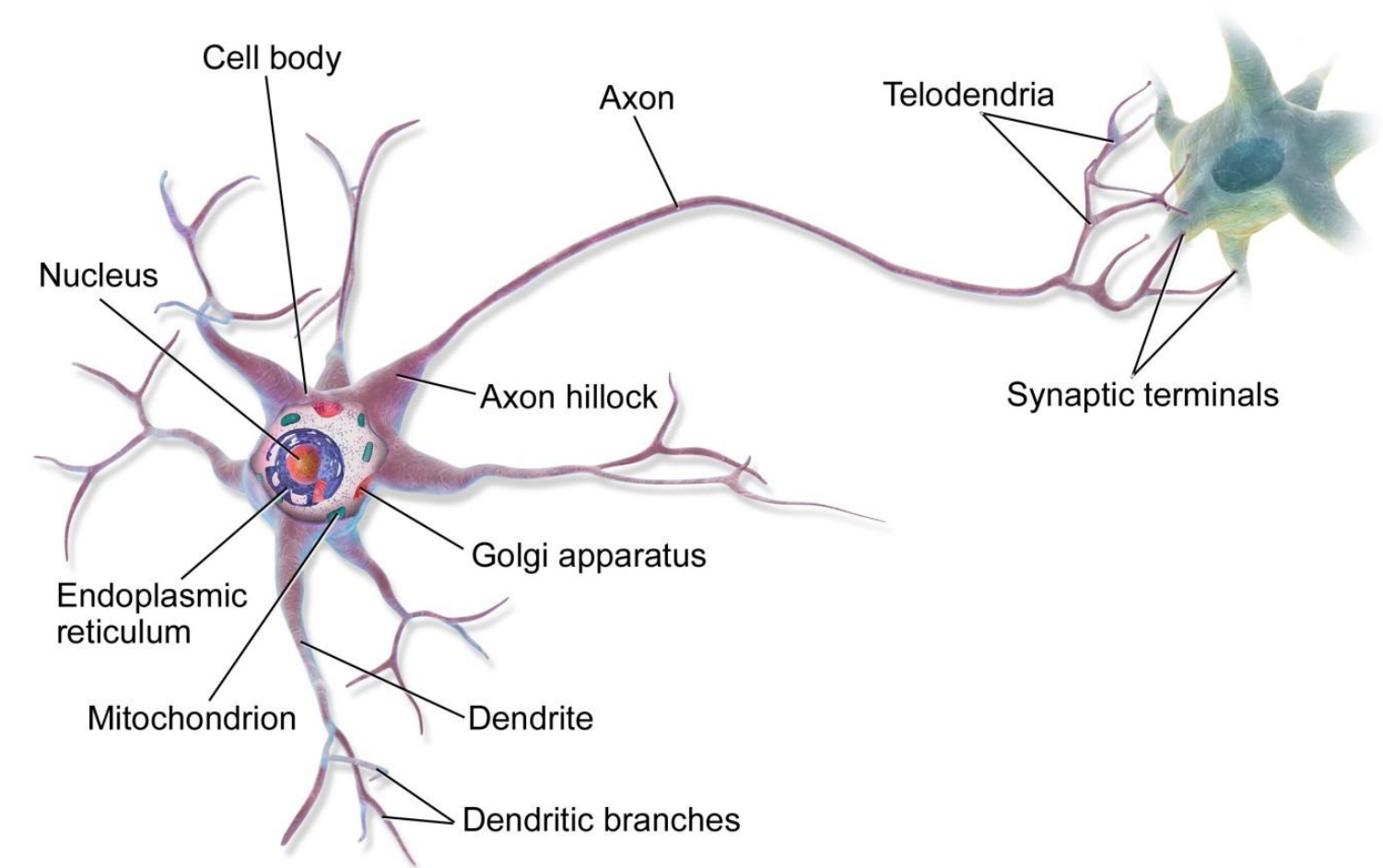
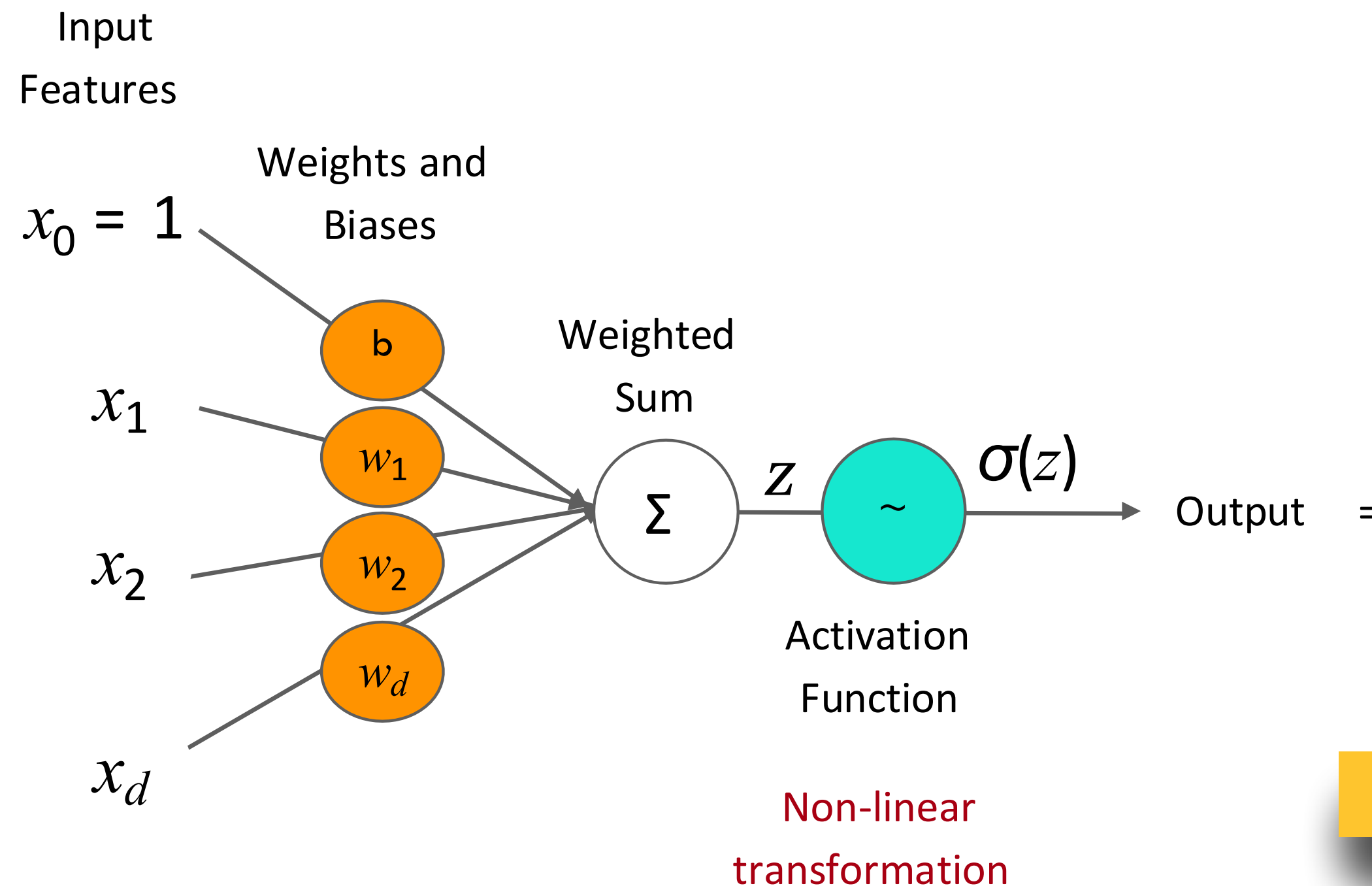
- HW1 release – Sep 15
- HW and late days: DO NOT email, your submission time will determine how many late days you used.
 - NO partial late days - 1 day = 24 hours!
- We will need you to submit all your code as a final deliverable. **PLAGIARISM will be strictly penalized**

Lecture Outline

- Announcements
- Feedforward Neural Net Language Models
- FFNN for Classification
- Training FFNNs
- Computation Graph and Backdrop

Neural Network Unit

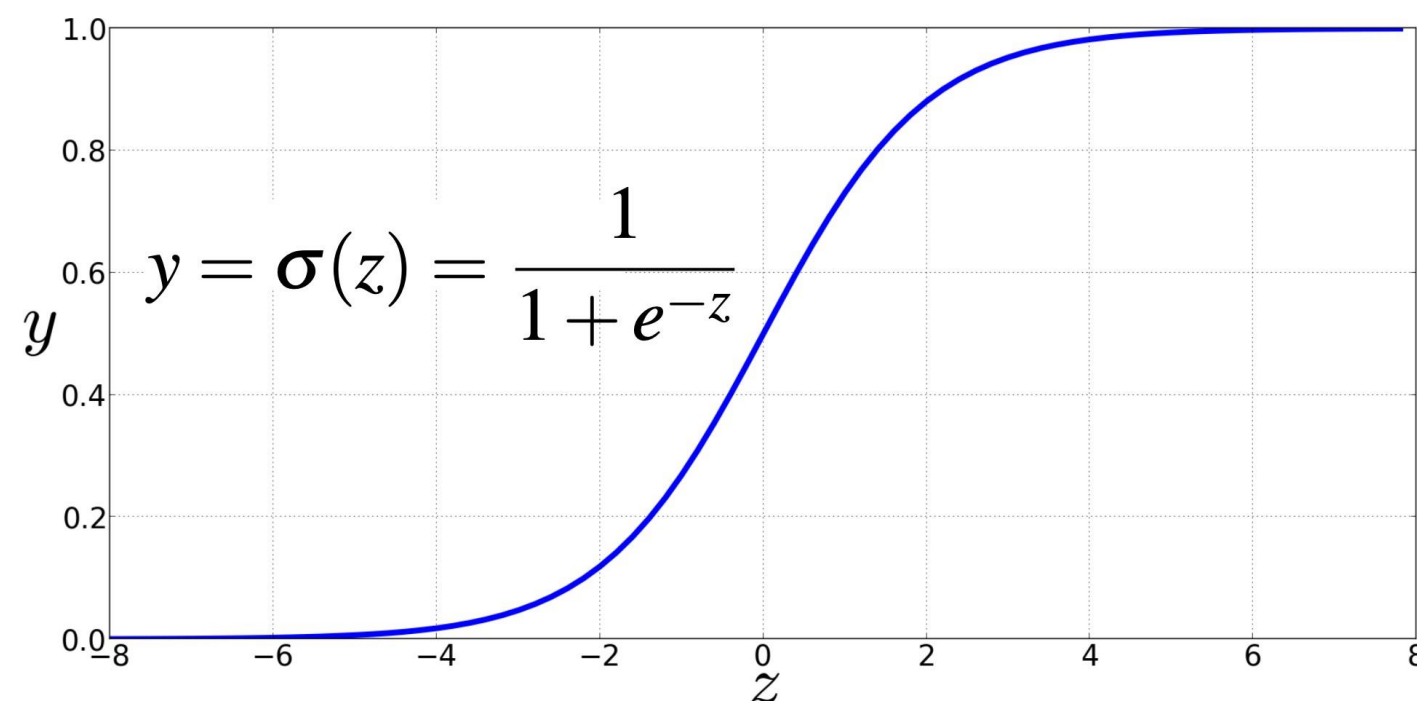
Logistic Regression is a very simple neural network



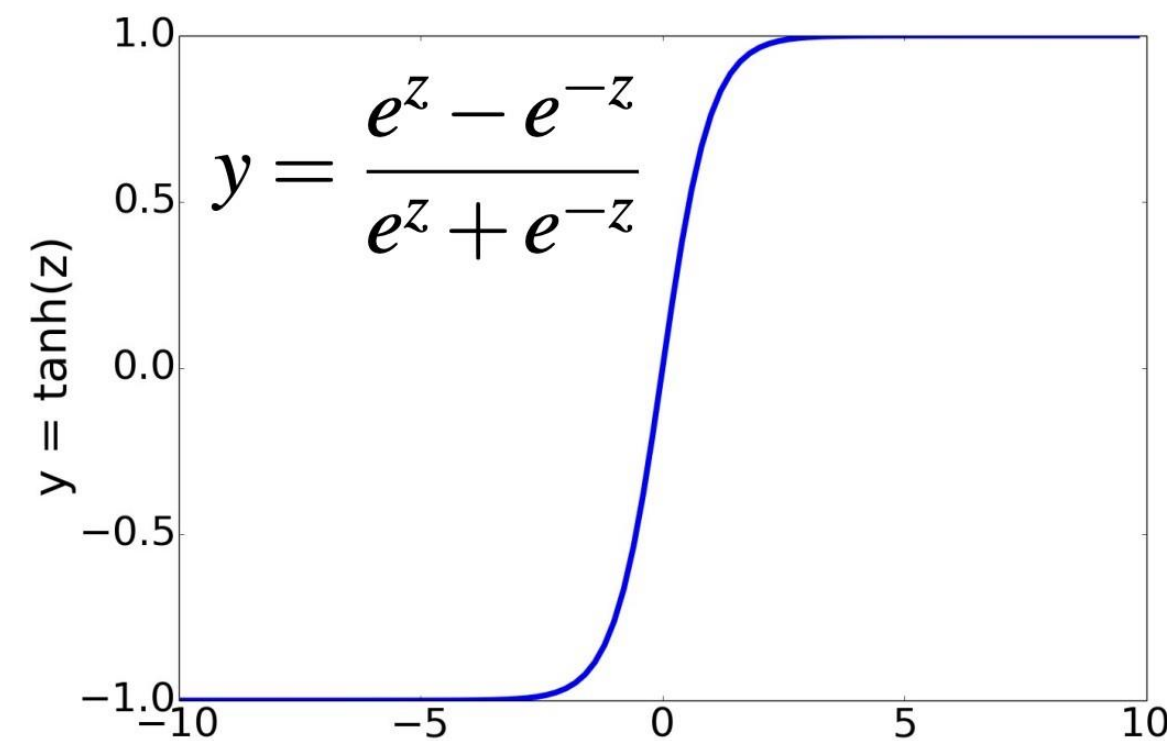
Resembles a neuron in the brain!

Non-Linear Activation Functions

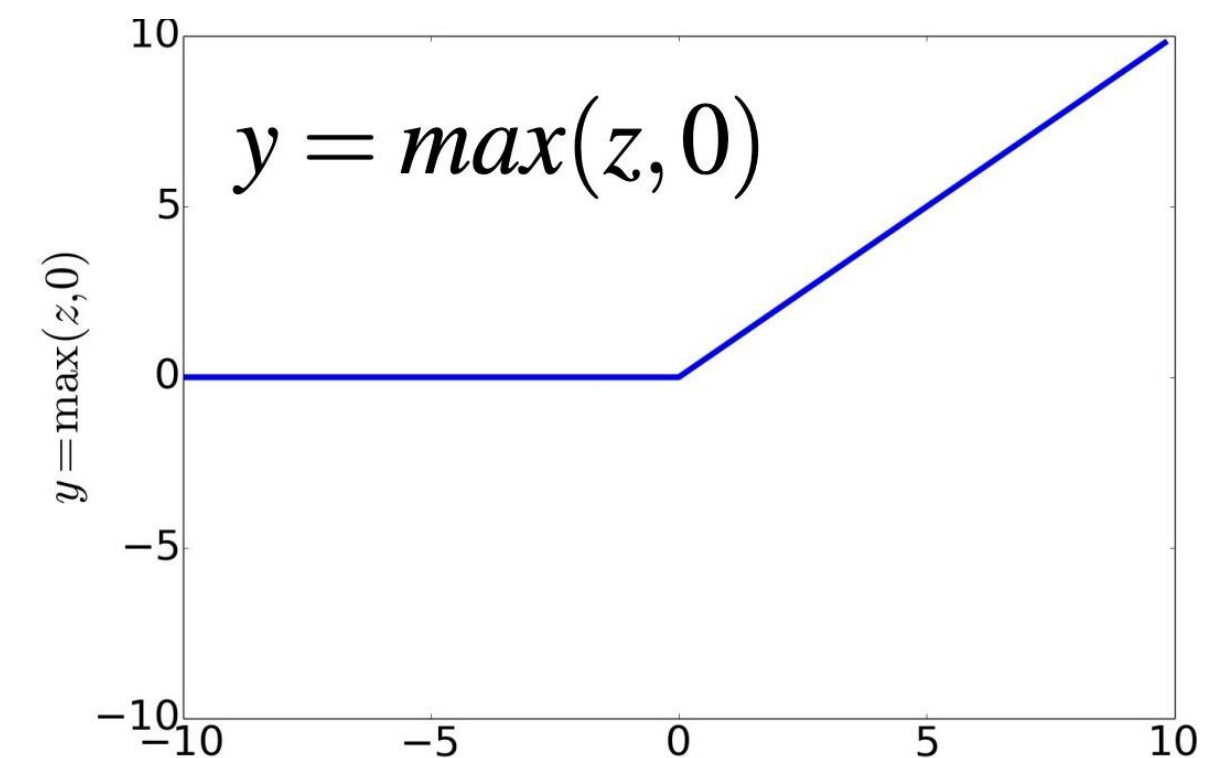
The key ingredient of a neural network is the non-linear activation function



sigmoid



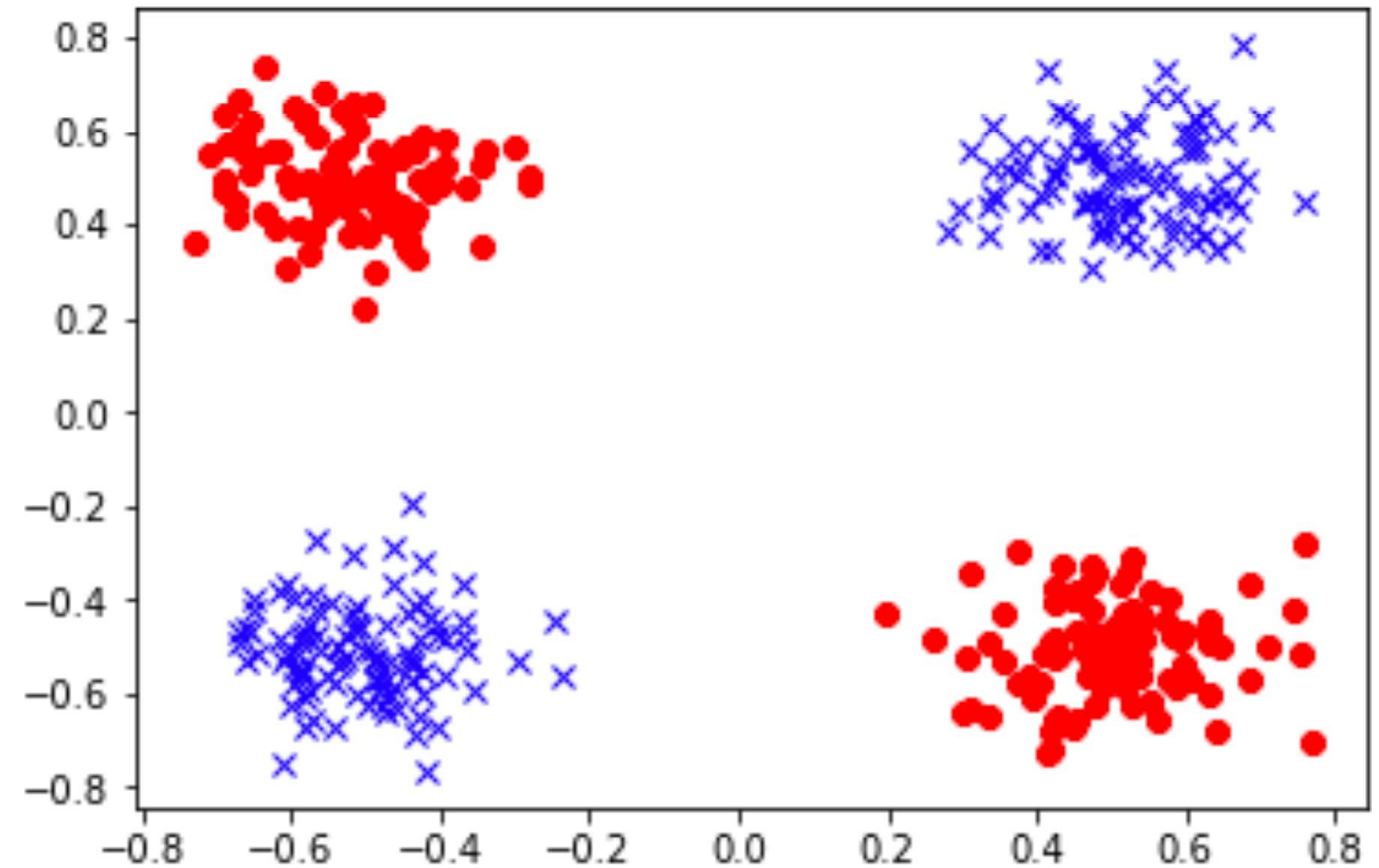
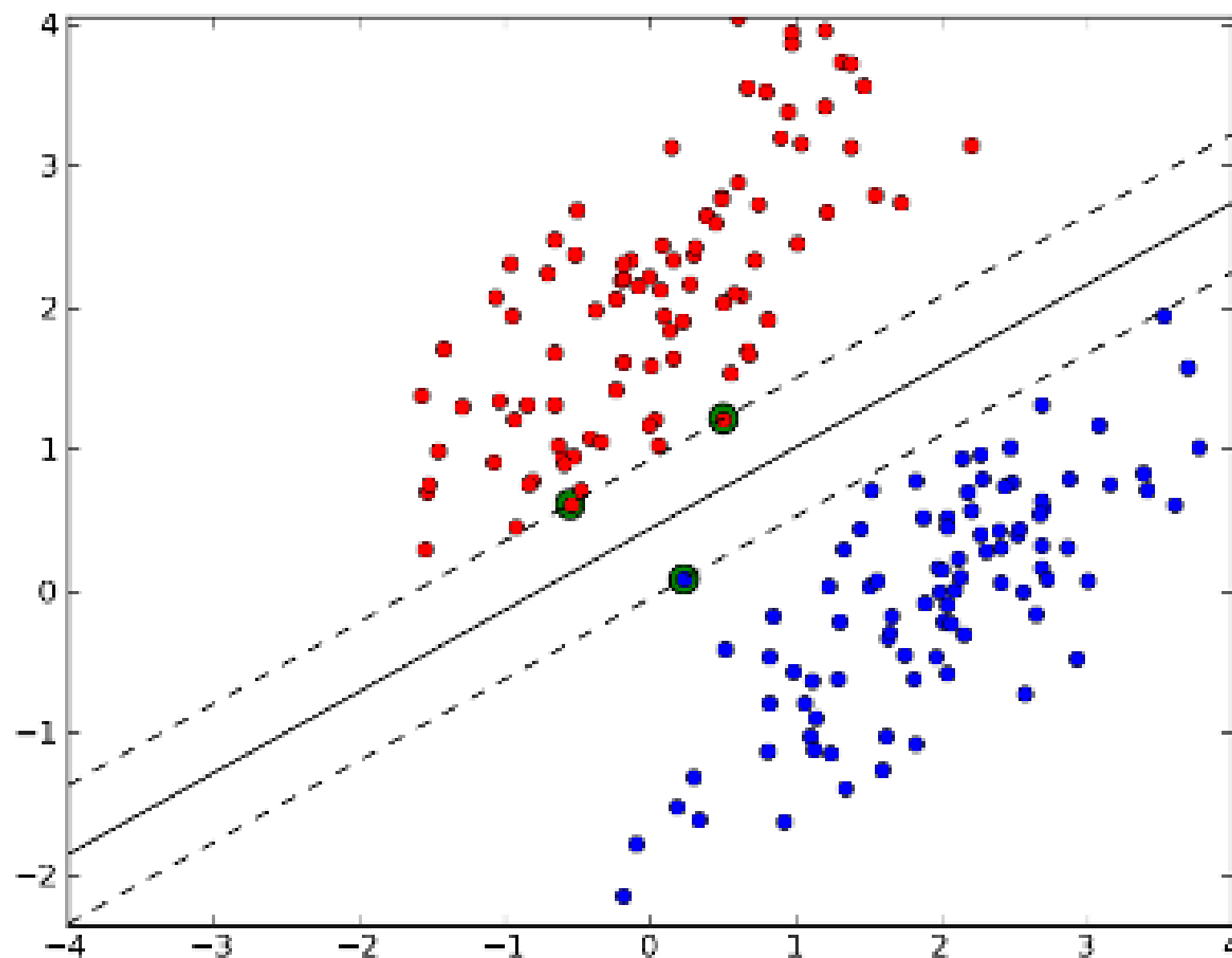
tanh



relu (Rectified Linear Unit)

Most common!

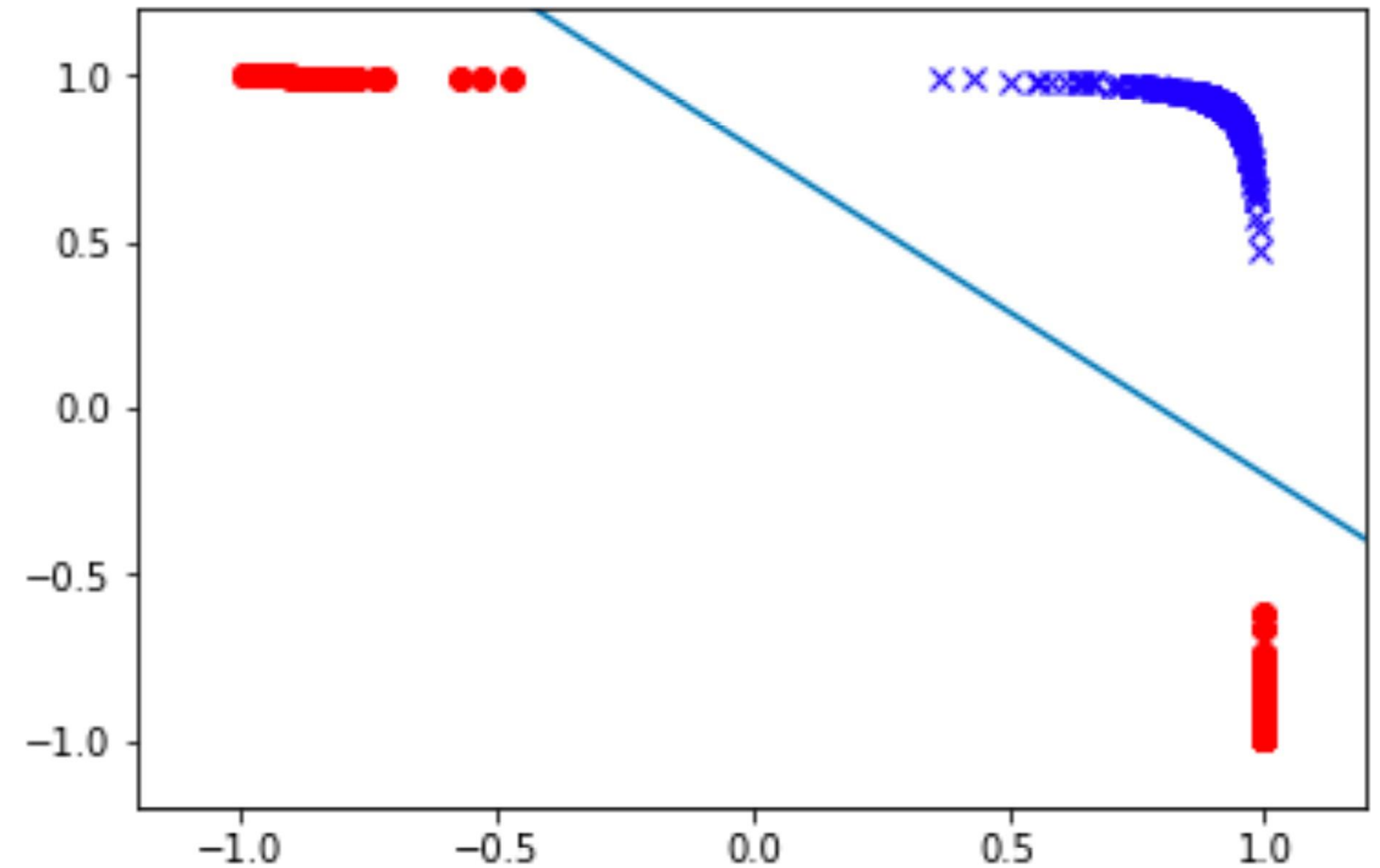
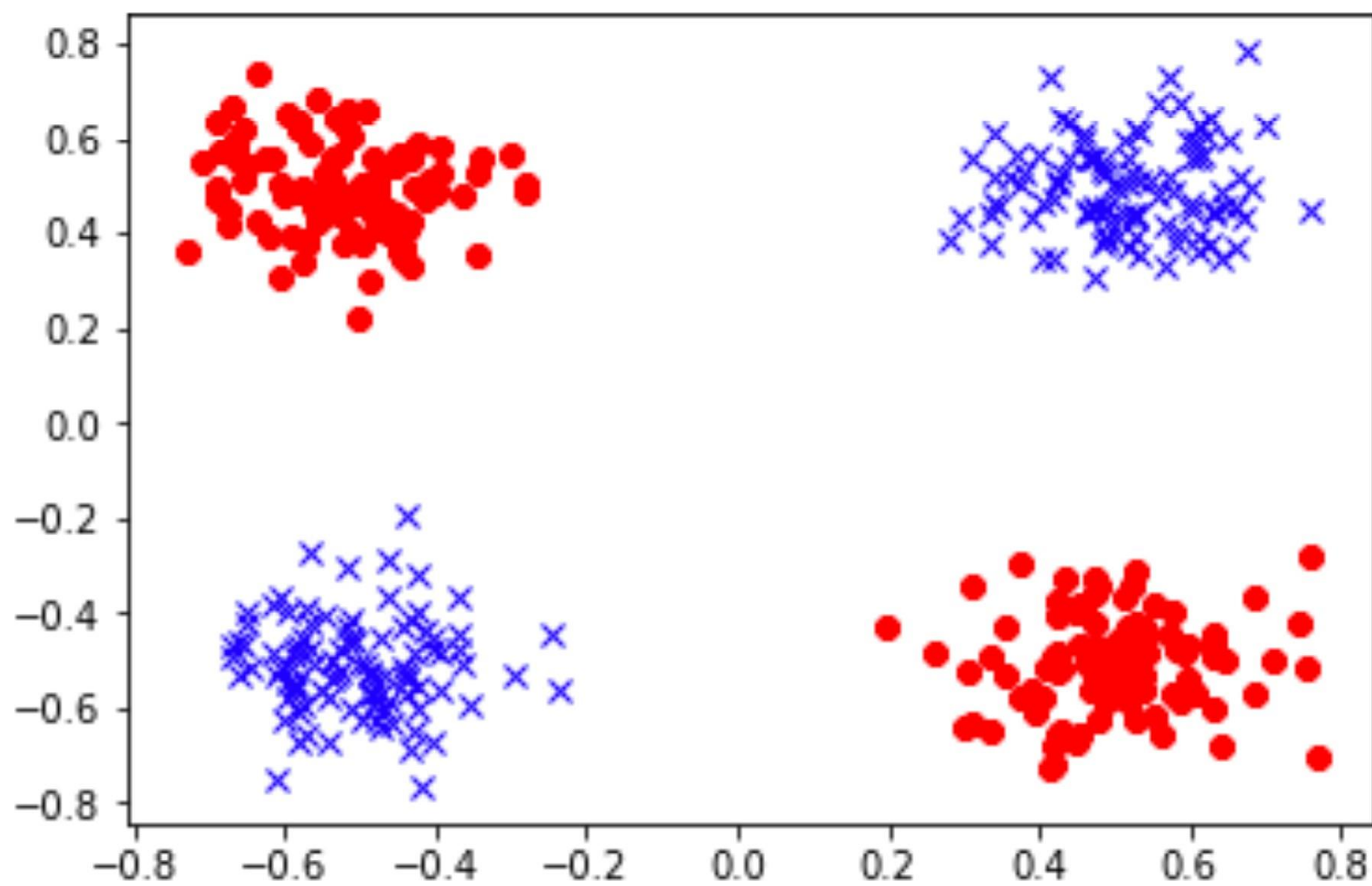
Linear vs. Non-linear Functions



Linearly inseparable

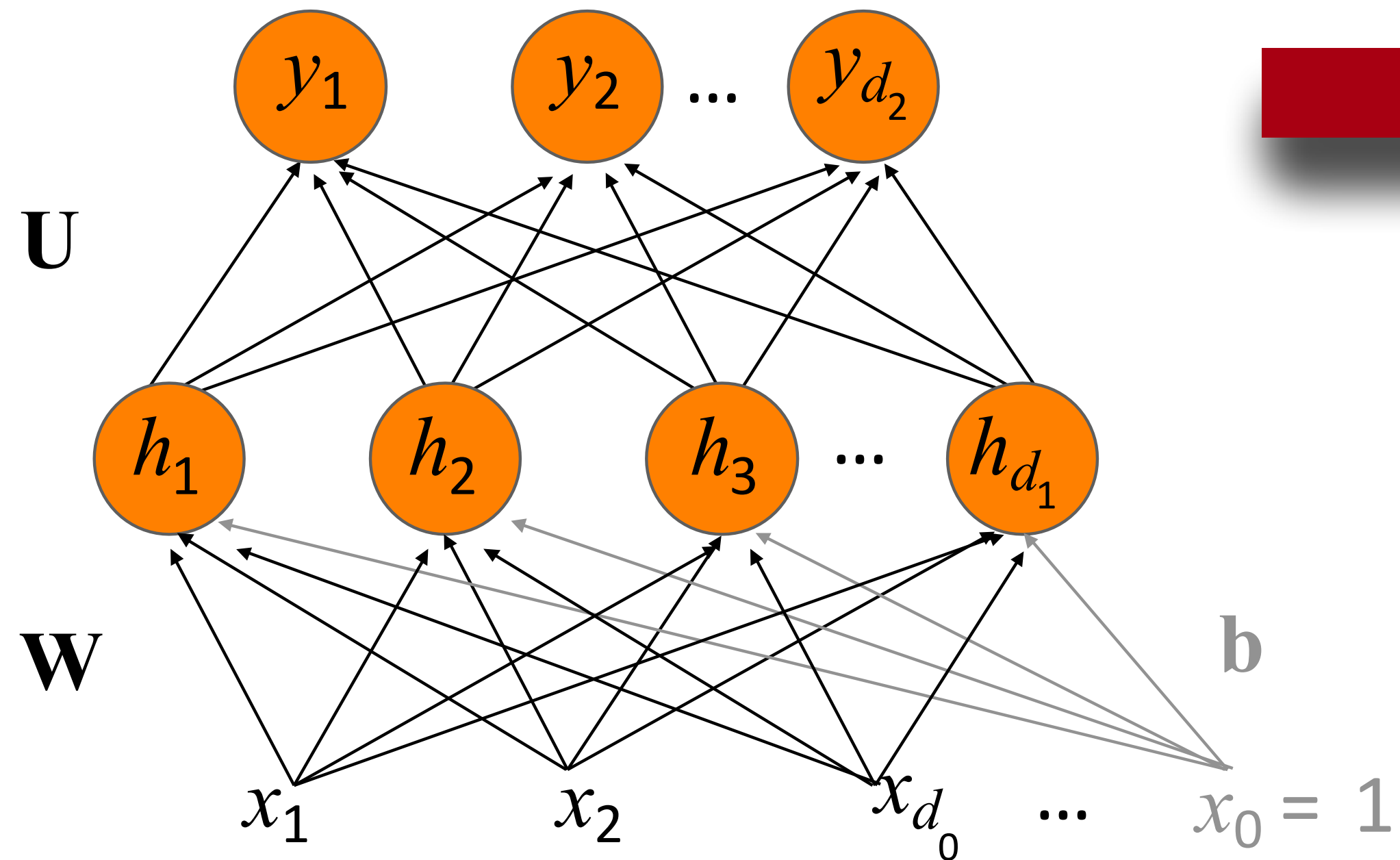
Power of non-linearity

After a $\tanh(\cdot)$ transformation:



Feedforward Neural Nets

Technically, can
learn any function!



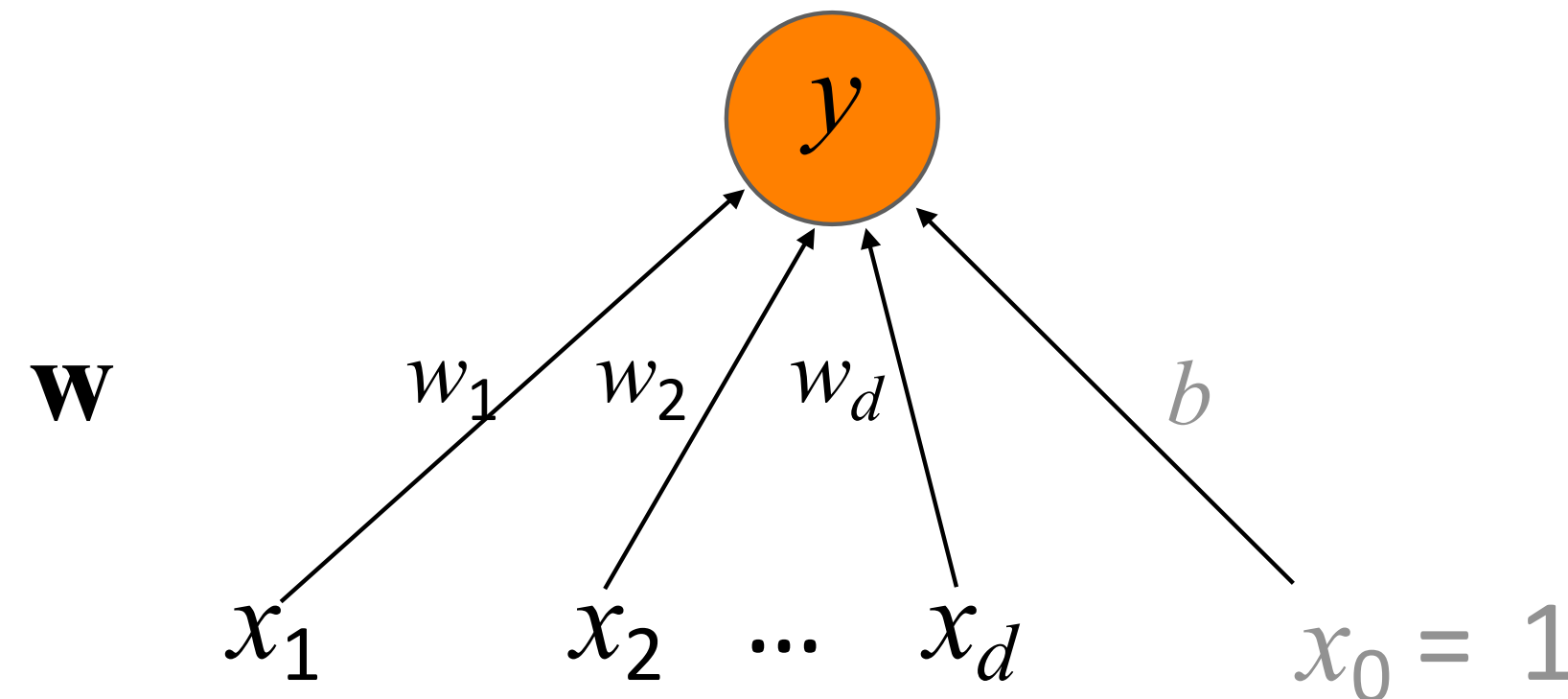
Multilayer Perceptron

Let's break it down by revisiting our logistic regression model

Binary Logistic Regression

Output layer: $y = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$

Input layer: vector



Weighted sum of all incoming, followed by a non-linear activation

1-layer Network

Don't count the input layer in counting layers!

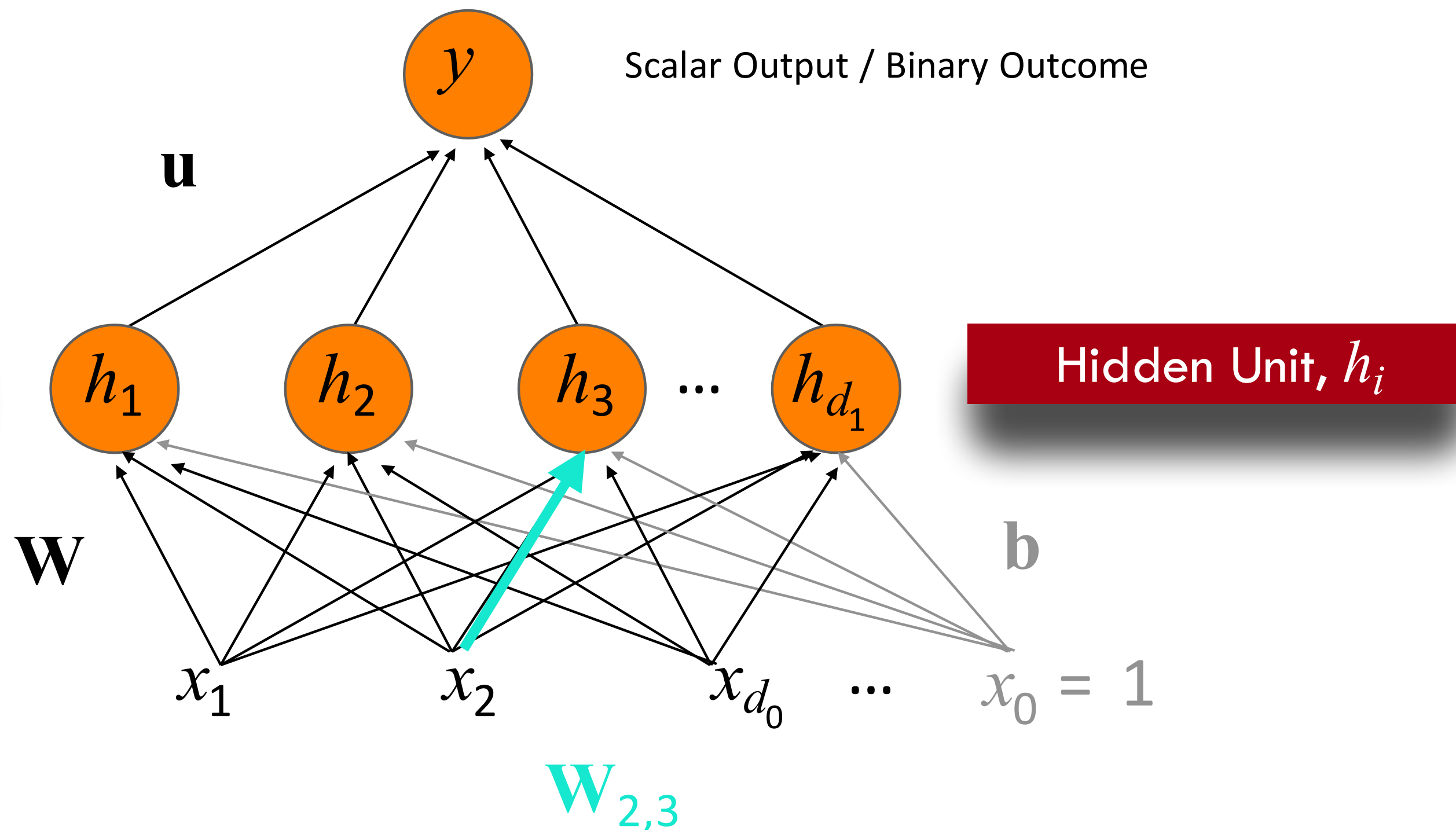
Two-layer Feedforward Network

Output layer: $y = \sigma(\mathbf{u}\mathbf{h})$

Hidden layer: $\mathbf{h} = g(\mathbf{W}\mathbf{x} + \mathbf{b})$

Usually ReLU or tanh

Input layer: vector



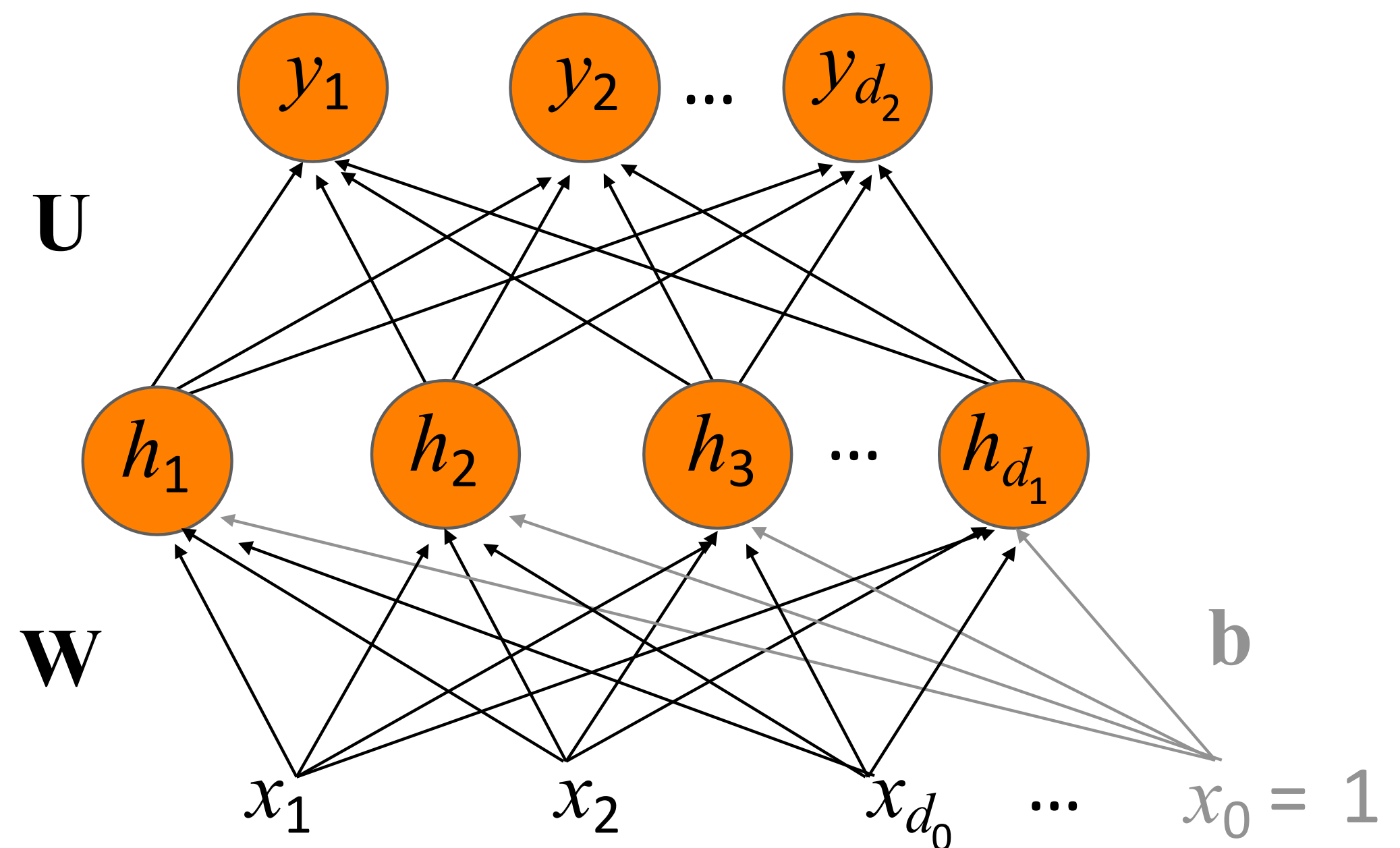
Two-layer Feedforward Network with Softmax Output

Output layer: $\mathbf{y} = \text{softmax}(\mathbf{U} \cdot \mathbf{h})$

Hidden layer: $\mathbf{h} = g(\mathbf{W}\mathbf{x} + \mathbf{b})$

Usually ReLU or tanh

Input layer: vector



What is $\mathbf{y}$?

Simpler Notification

Replacing the bias unit

Instead of:

$$x = x_1, x_2, \dots, x_{n_0}$$

$$h = \sigma(Wx + b)$$

$$h_j = \sigma \left(\sum_{i=1}^{n_0} W_{ji} x_i + b_j \right)$$

We'll do this:

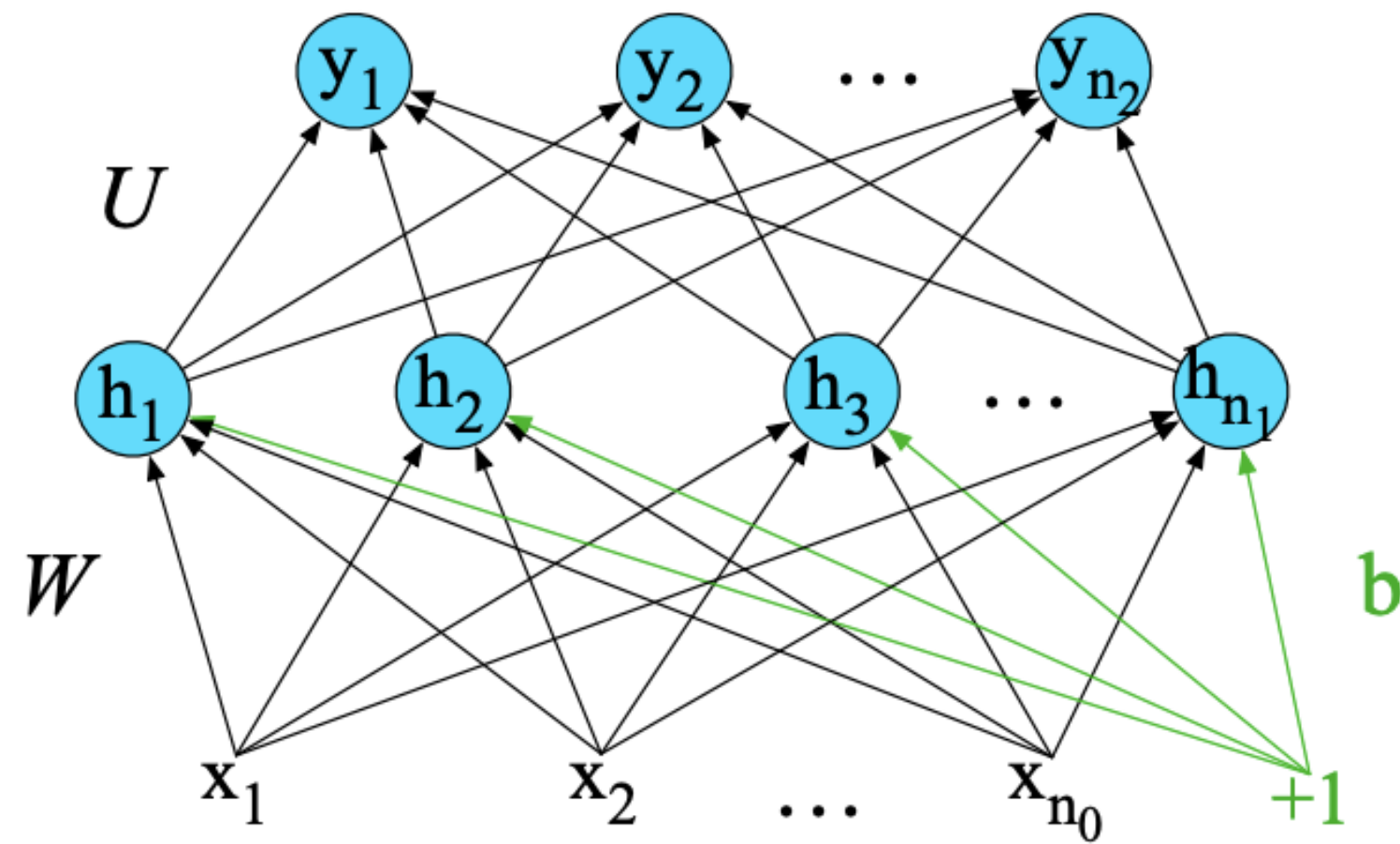
$$x = x_0, x_1, x_2, \dots, x_{n_0}$$

$$h = \sigma(Wx)$$

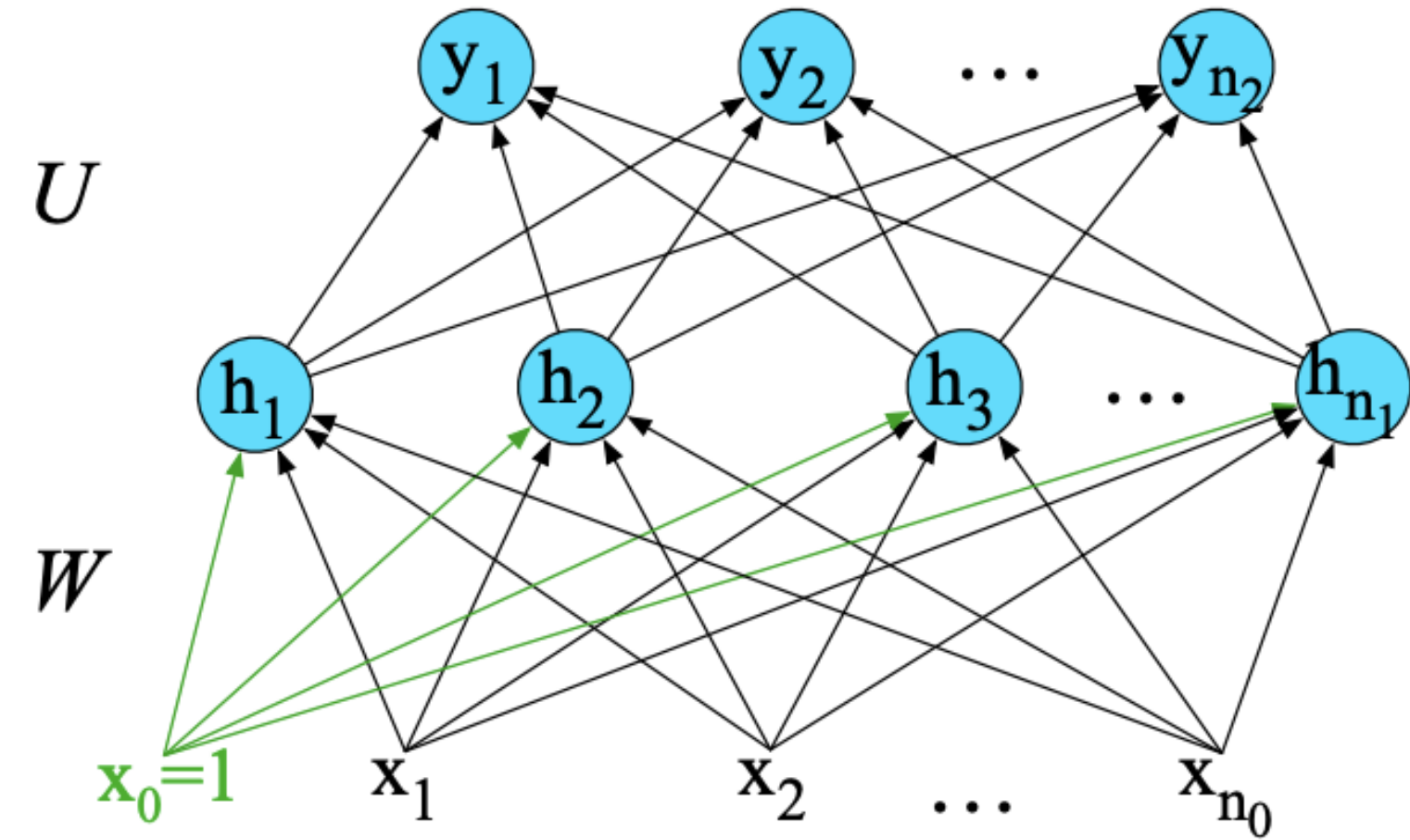
$$\sigma \left(\sum_{i=0}^{n_0} W_{ji} x_i \right)$$

Simpler Notification

Instead of:



We'll do this:



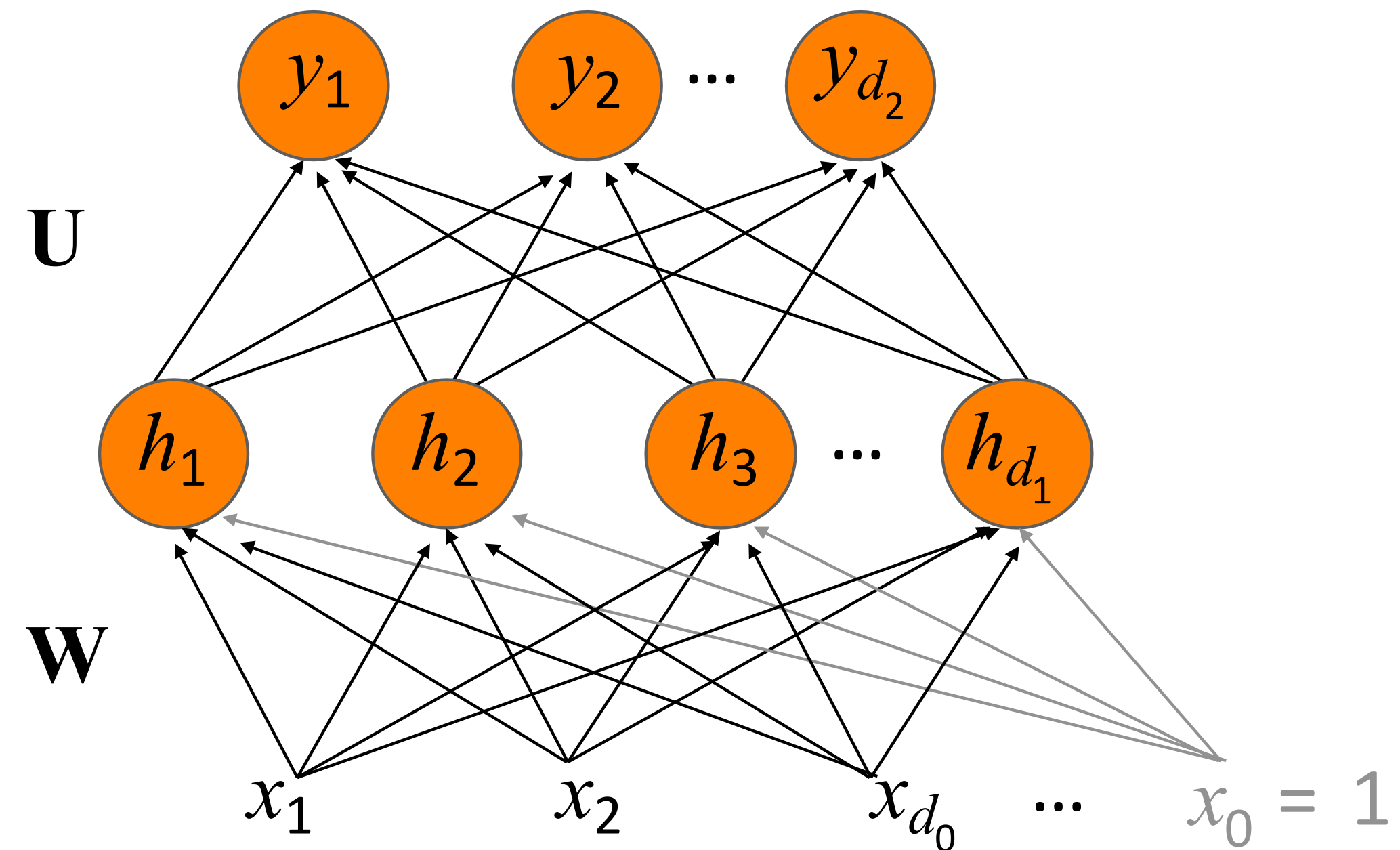
Two-layer FFNN: Notation

Output layer: $\mathbf{y} = \text{softmax}(\mathbf{U} \cdot \mathbf{h})$

Hidden layer: $\mathbf{h} = g(\mathbf{W}\mathbf{x}) = g\left(\sum_{i=1}^{d_0} \mathbf{W}_{ji} \mathbf{x}_i\right)$

Usually ReLU or tanh

Input layer: vector



We usually drop the $\mathbf{b}$ and add one dimension to the $\mathbf{W}$ matrix

Use Case for FFNN

Let's consider 2 (simplified) tasks:

- Text classification
- Language Modeling

Text Classification: Sentiment Analysis

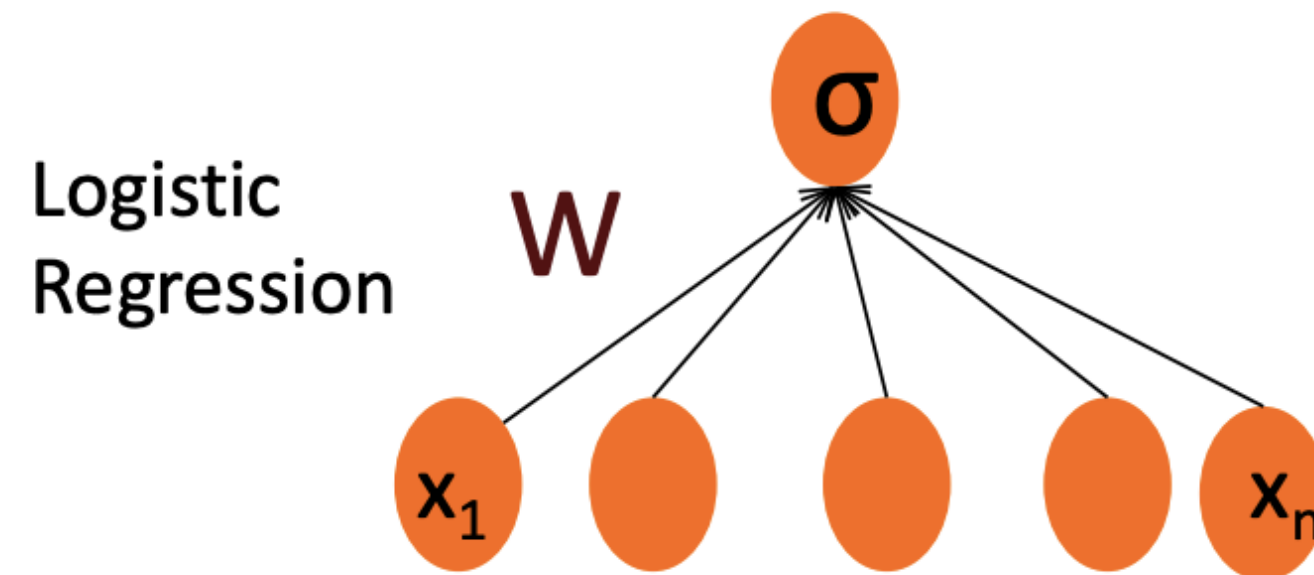
We could do exactly what we did with logistic regression

- Input layer are binary features as before
- Output layer is 0 or 1

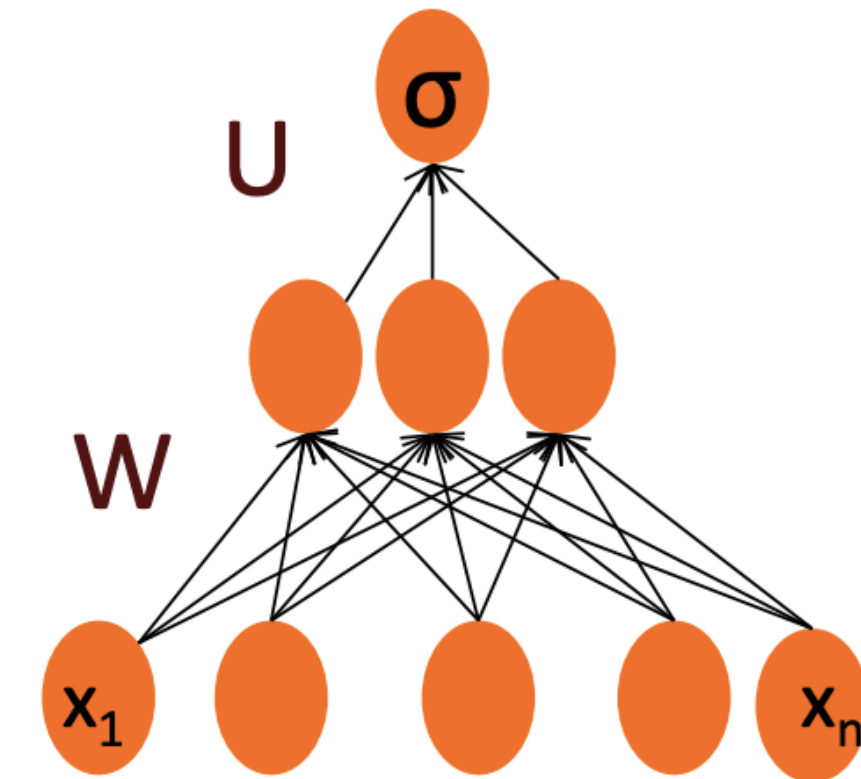
Sentiment Features

Var	Definition
x_1	count(positive lexicon) $\in$ doc)
x_2	count(negative lexicon) $\in$ doc)
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	log(word count of doc)

FFNN for Sentiment Analysis



2-layer
feedforward
network



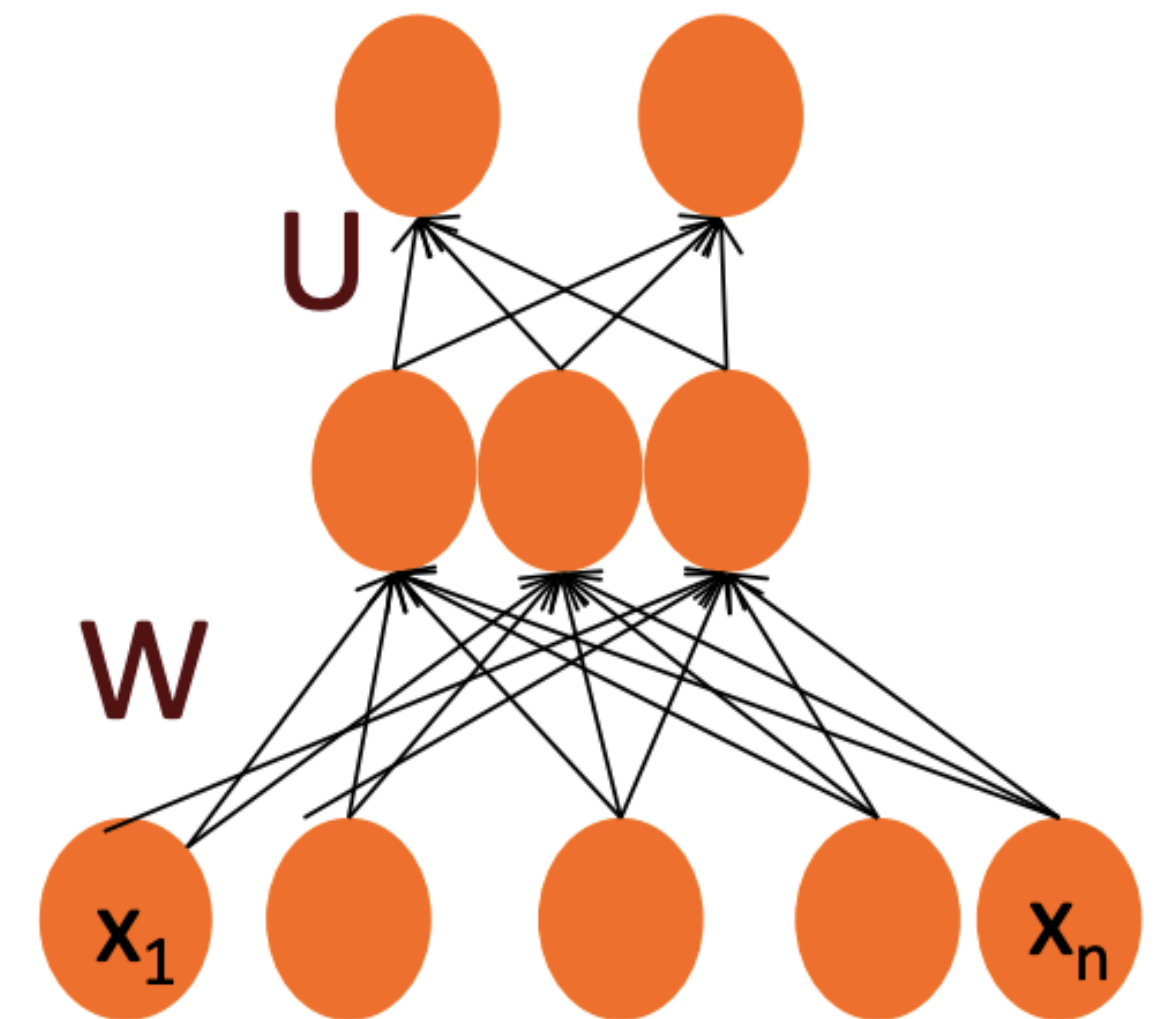
Just adding a hidden layer to logistic regression

- allows the network to use non-linear interactions between features
- which may (or may not) improve performance.

Multiclass Outputs

What if you have more than two output classes?

- Add more output units (one for each class)
- And use a “softmax layer”



Multiclass Outputs

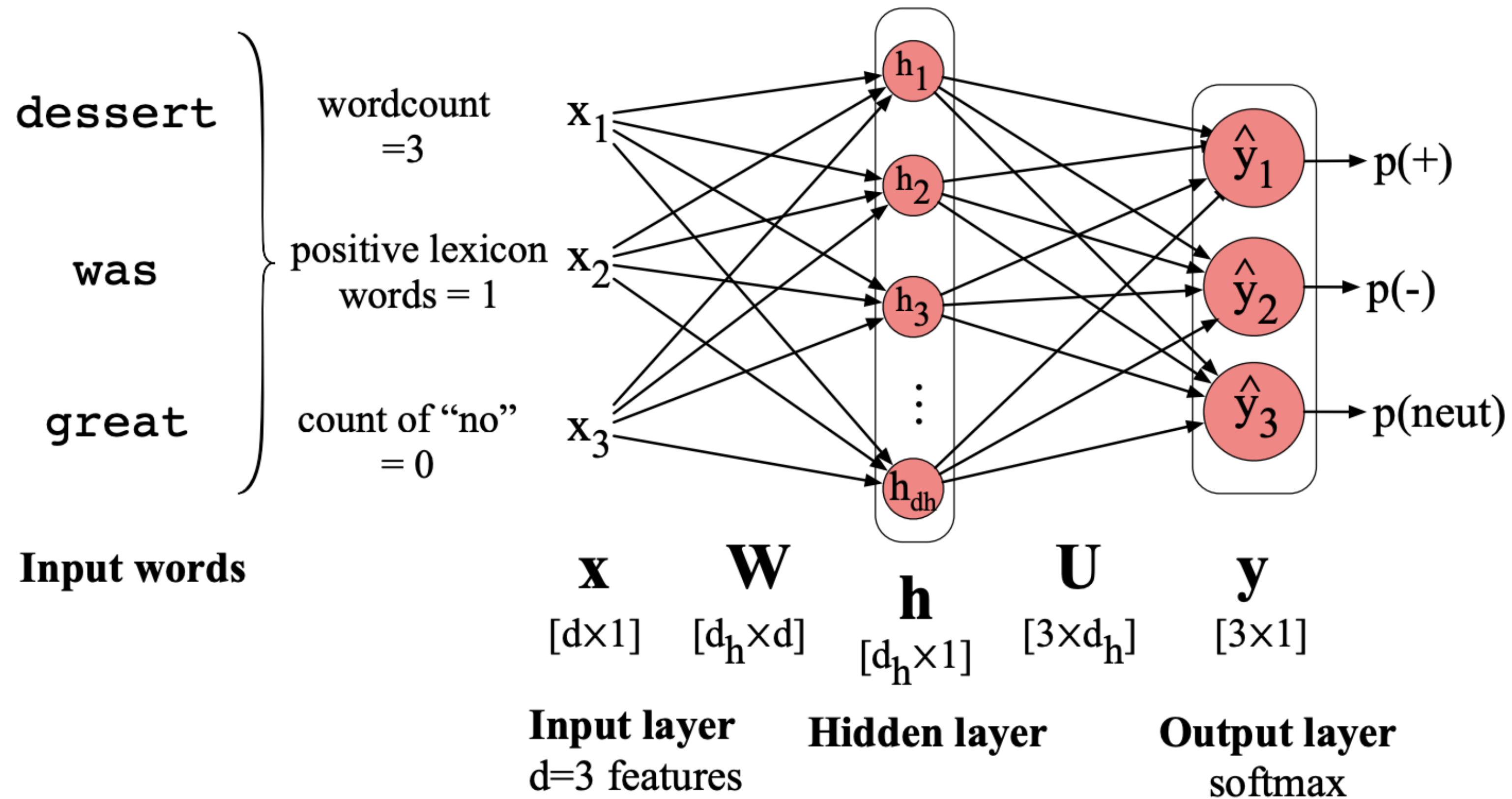
$\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_d]$ (each $\mathbf{x}_i$ is a hand-designed feature)

$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$

$\mathbf{z} = \mathbf{U}\mathbf{h}$

$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$

Sentiment Analysis



Vectoring for Parallel Computing

We would like to efficiently classify the whole test set of m observations.

So we vectorize: pack all the input features into X

- Each row $x^{(i)}$ of X is a row vector with all the features for example $x^{(i)}$
- Feature dimensionality is d , X is $[m \times d]$

Vectoring for Parallel Computing

Each input is now a row vector

- $\mathbf{X}$ is of shape $[m \times d]$
- $\mathbf{W}$ is of shape $[d_h \times d]$
- We'll need to do some reordering and transposing
- Bias vector $\mathbf{b}$ that used to be $[1 \times d_h]$ is now $[m \times d_h]$

$$\mathbf{H} = \sigma(\mathbf{XW}^T + \mathbf{b})$$

$$\mathbf{Z} = \mathbf{HU}^T$$

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{Z})$$

Even better: representation learning

The real power of deep learning comes from the ability to learn features from the data

Instead of using hand-built human-engineered features for classification

Use learned representations like embeddings!

Embedding Matrix: E

- An embedding is a vector of dimension $[1 \times d]$ that represents the input token
- An embedding matrix E is a dictionary, one row per token of vocab V
- E has shape $[|V| \times d]$
- Embedding matrices are central to NLP; they represent input text in LLMs and all NLP tools

Text Classification from Embeddings

Given tokenized input: dessert was great

Select the embedding vectors from E:

1. Convert tokens into vocabulary indices
 - $w = [3, 524, 902]$
2. Use indexing to select the corresponding rows from E
 - row 3, row 524, row 902

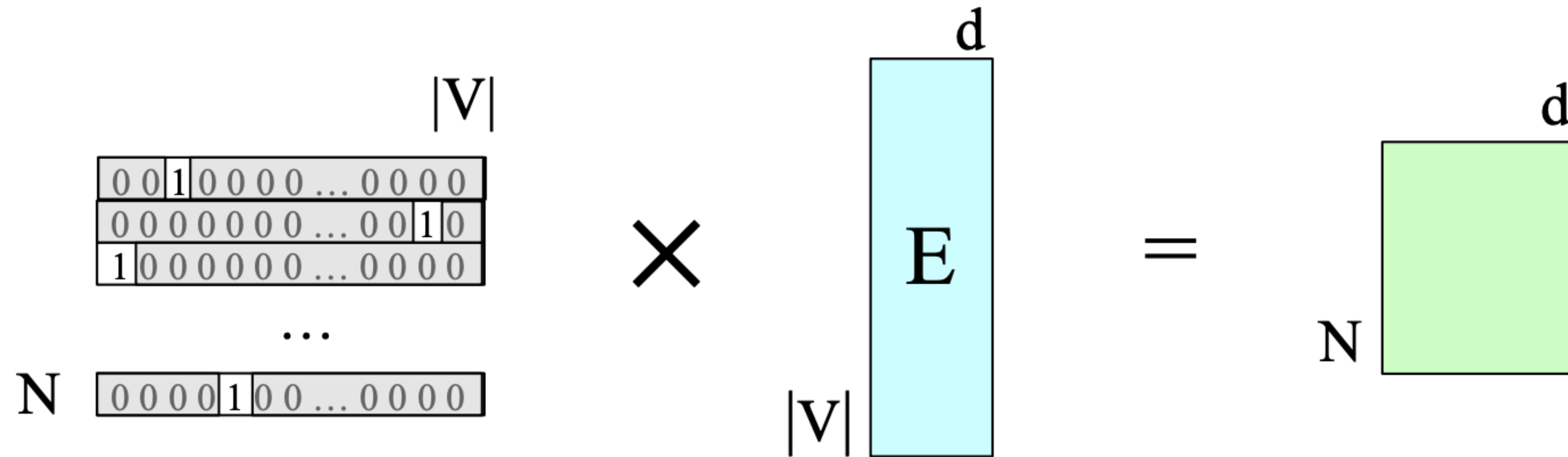
Another way to think of indexing from E

- Treat each input word as one-hot vector
 - If **dessert** is index 3:
$$\begin{matrix} [0 & 0 & 1 & 0 & 0 & 0 & 0 & \dots & 0 & 0 & 0 & 0] \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & \dots & \dots & |V| \end{matrix}$$
- Multiply it by **E** to select out the embedding for **dessert**

$$\begin{array}{c} 1 \\ \hline 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad \dots \quad 0 \quad 0 \quad 0 \quad 0 \\ \hline \end{array} \begin{matrix} 3 \\ |V| \end{matrix} \times \begin{array}{c} d \\ \hline 3 \\ \hline E \\ \hline |V| \end{array} = \begin{array}{c} 1 \\ \hline \\ \hline \end{array} \begin{matrix} d \end{matrix}$$

The diagram illustrates the matrix multiplication of a one-hot vector with an embedding matrix E. The one-hot vector has a dimension of |V| and a value of 1 at index 3. The embedding matrix E has dimensions |V| by d. The result is a vector of dimension d, where the value at index 3 is the embedding for the word 'dessert'.

Collecting Embeddings from E for N words



Given window of N tokens, represented by $N [1 \times d]$ embeddings

Need to return a single class (e.g., pos or neg)

Text comes in different lengths

Given window of N tokens, represented by $N [1 \times d]$ embeddings

Need to return a single class (e.g., pos or neg)

- Concatenate all the inputs into one long vector of shape $[1 \times dN]$, i.e., input is length N of longest review
 - If shorter, then pad with zero embedding
 - Truncate if you get longer reviews at test time
- Pool inputs into a single short $[1 \times d]$ vector. A single “sentence embedding” (the same dimensionality as a word) to represent all words. Less info, but very efficient and fast

Pooling

- Intuition: exactly position not so important for sentiment
 - We will just do some sort of averaging of all vectors.

- **Mean pooling**
$$\mathbf{x}_{mean} = \frac{1}{N} \sum_{i=1}^N \mathbf{e}(w_i)$$

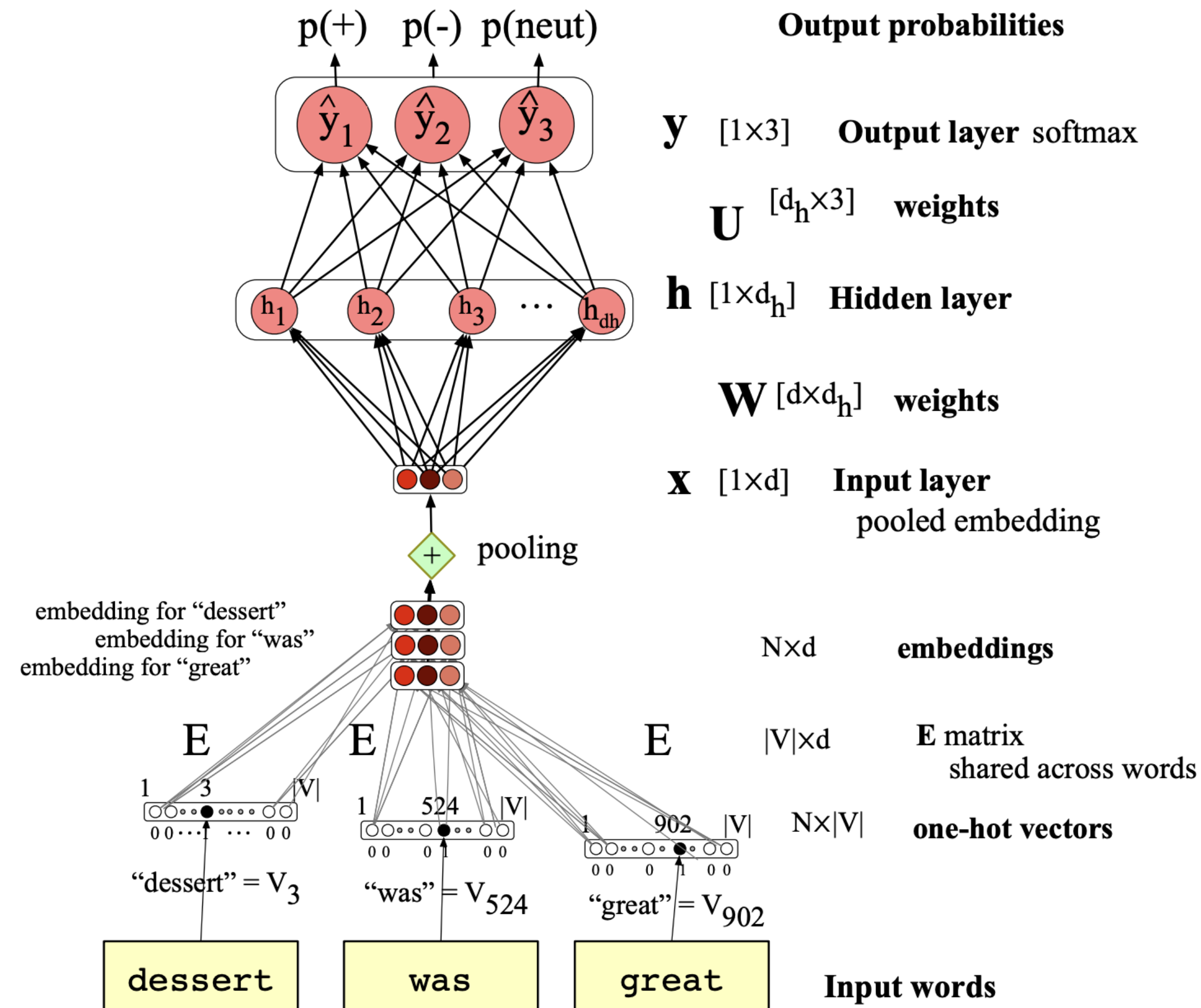
$$\mathbf{x} = \text{mean}(\mathbf{e}(w_1), \mathbf{e}(w_2), \dots, \mathbf{e}(w_n))$$

$$\mathbf{h} = \sigma(\mathbf{x}\mathbf{W} + \mathbf{b})$$

$$\mathbf{z} = \mathbf{h}\mathbf{U}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

Pooling



FFNN Language Models

Feedforward Neural Language Models

- Language Modeling: Calculating the probability of the next word in a sequence given some history.
- Compared to n-gram language models, neural network LMs achieve much higher performance
 - In general, count-based methods can never do as well as optimization-based ones
- State-of-the-art neural LMs are based on more powerful neural network technology like Transformers
- But simple feedforward LMs introduce many important concepts!

Simple Feedforward Neural LMs

Task: predict next word w_t given prior words $w_{t-1}, w_{t-2}, w_{t-3}, \dots$

Problem: Now we are dealing with sequences of arbitrary length....

Solution: Sliding windows (of fixed length)

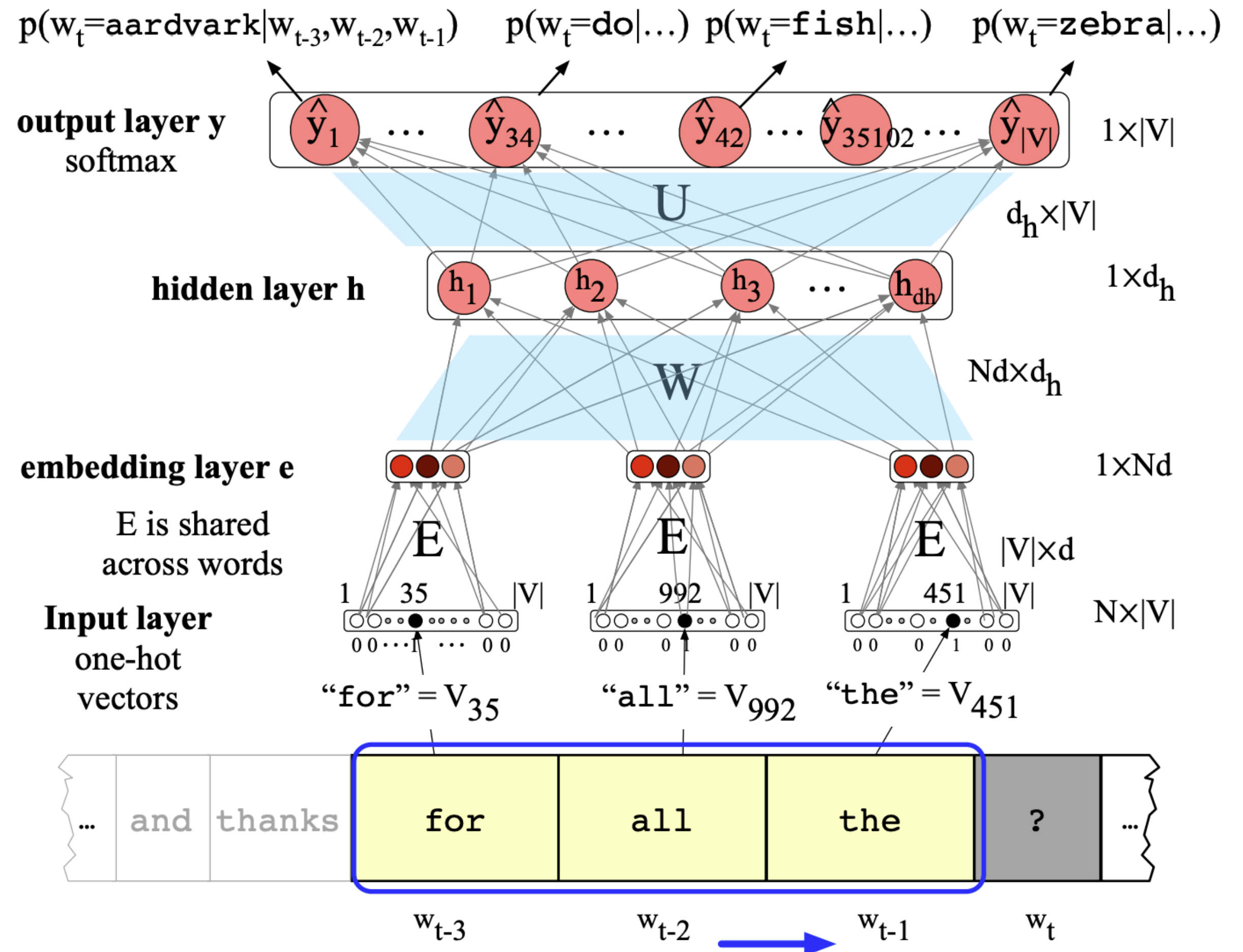
Basis of word embedding models!

$$P(w_t | w_{t-1}) \approx P(w_t | w_{t-1:t-M+1})$$

First introduced by Yoshua Bengio and colleagues in 2003

Feedforward Neural LM

- Sliding window of size 4 (including the target word)
- Every feature in the embedding vector connected to every single hidden unit
- Projection / embedding layer is a kind of input layer
 - This is where we plug in our word2vec embeddings
 - May or may not update embedding weights



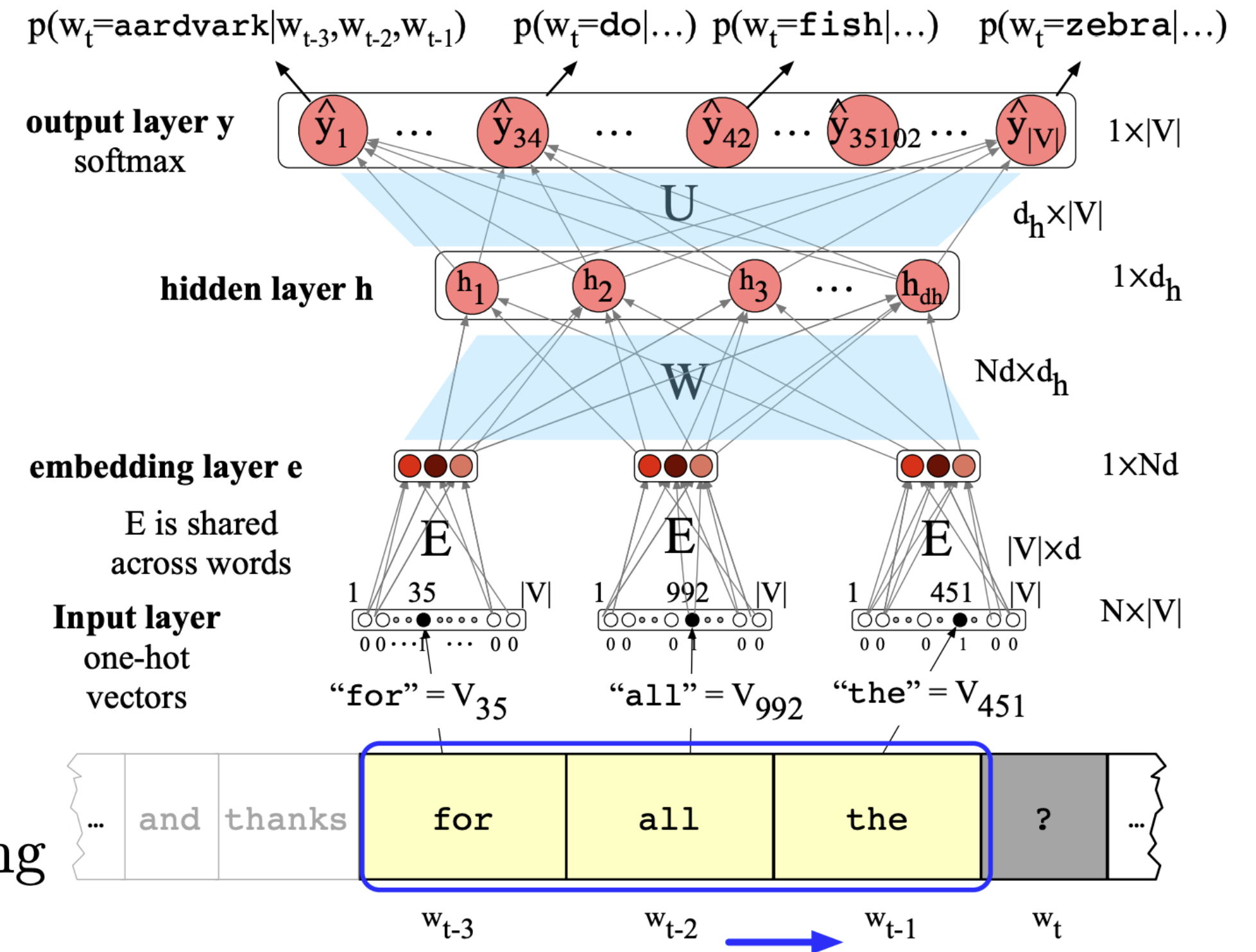
Feedforward Neural LM Equations

$$\mathbf{e} = [\mathbf{E}\mathbf{x}_{t-3}; \mathbf{E}\mathbf{x}_{t-2}; \mathbf{E}\mathbf{x}_{t-1}]$$

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{e} + \mathbf{b})$$

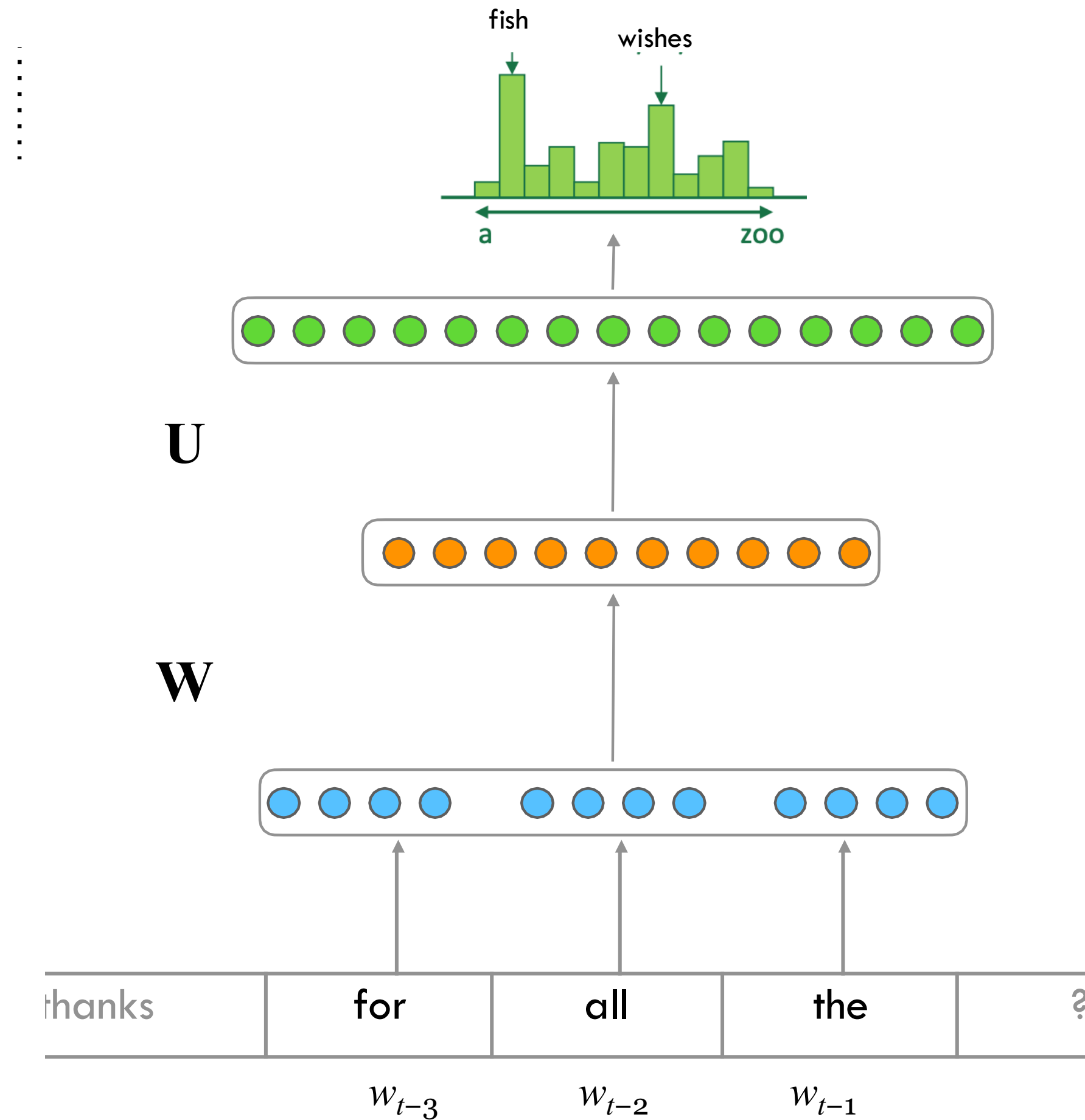
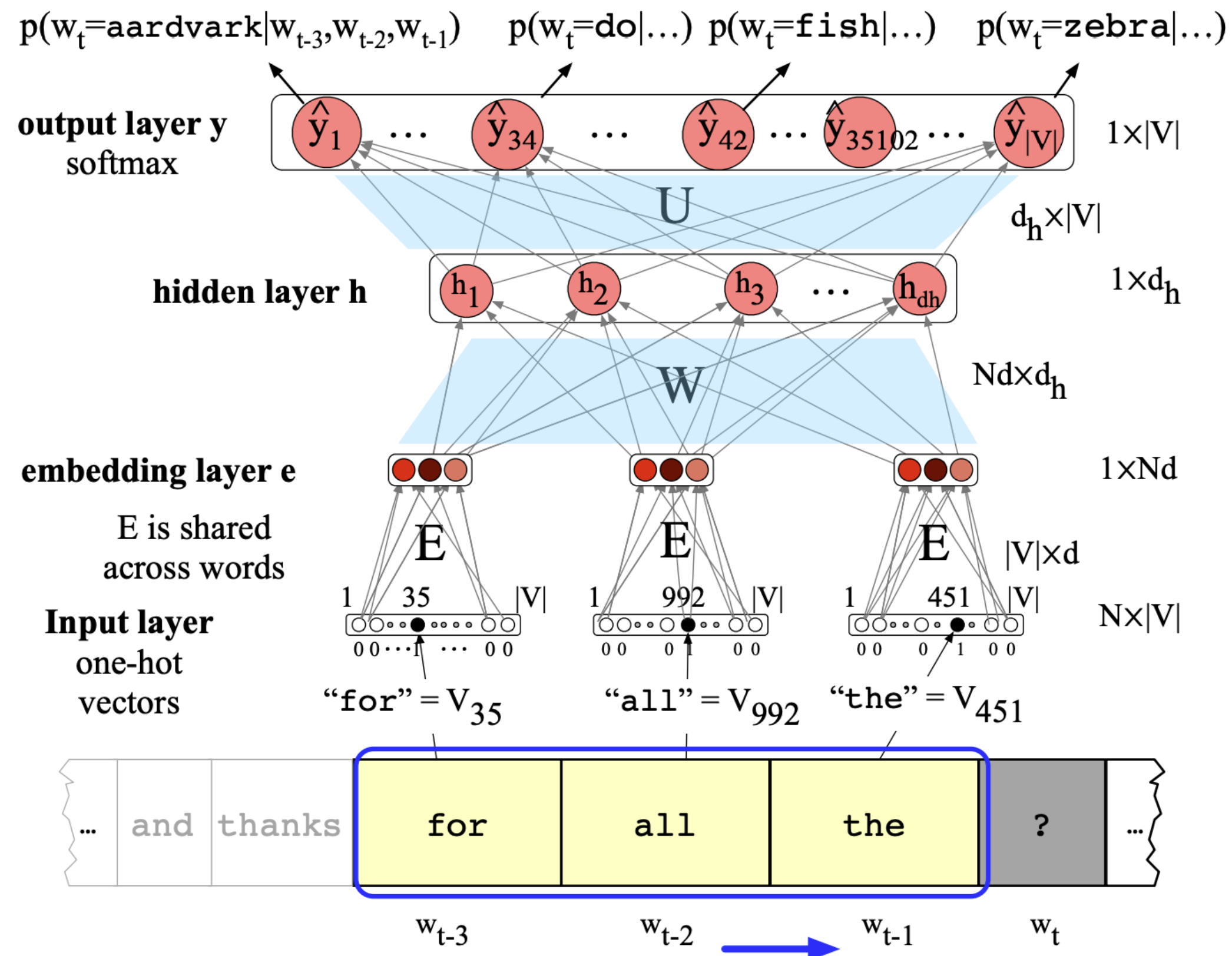
$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$



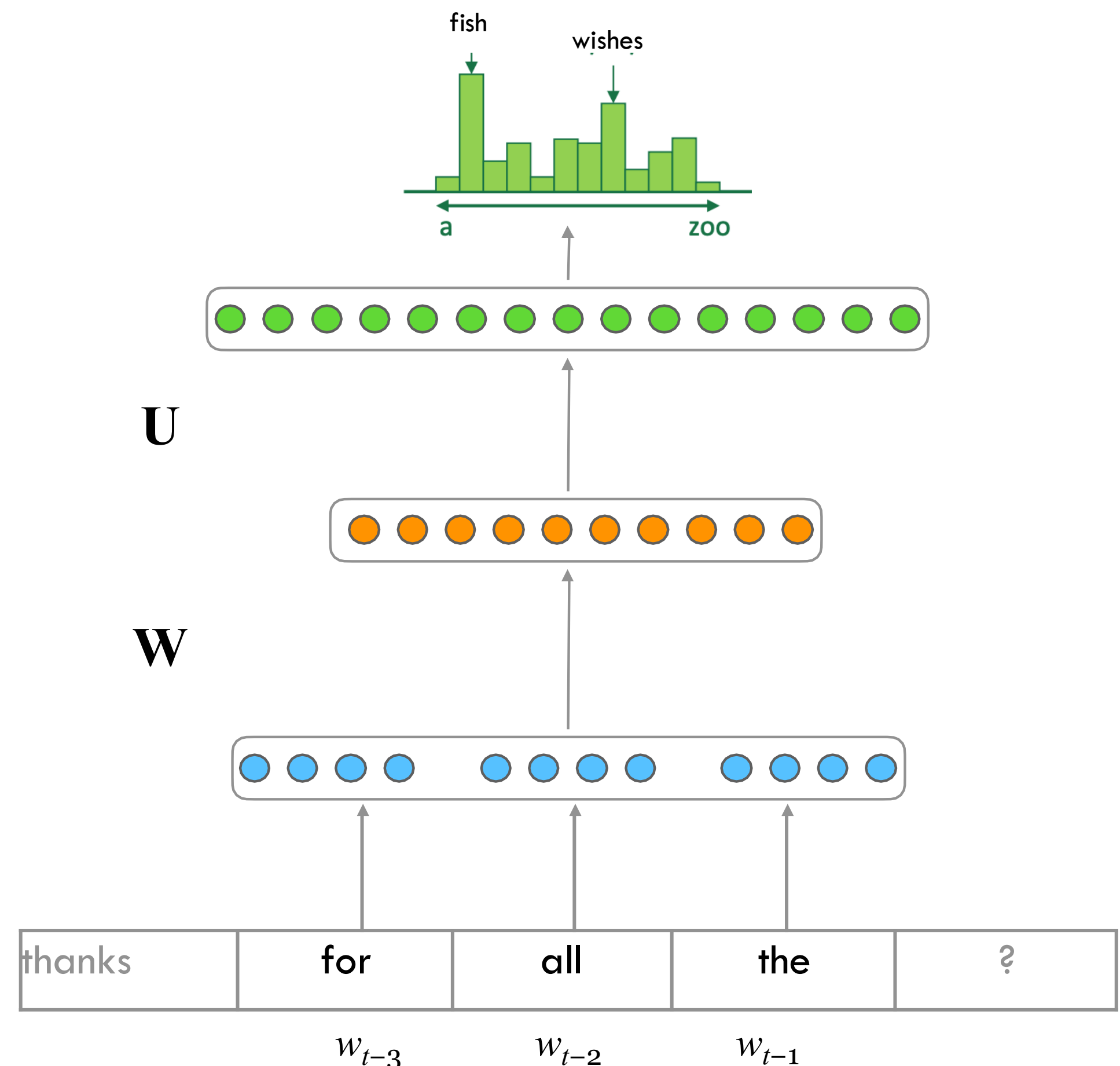
So $\hat{y}_{42}$ is the probability of the next word w_t being $V_{42} = \text{fish}$.

Simplified Representation



Feedforward LMs: Windows

- The goodness of the language model depends on the size of the sliding window!
- Fixed window can be too small
- Enlarging window enlarges $\mathbf{W}$
- Each word uses different rows of $\mathbf{W}$. We don't share weights across the window.
- Window can never be large enough!



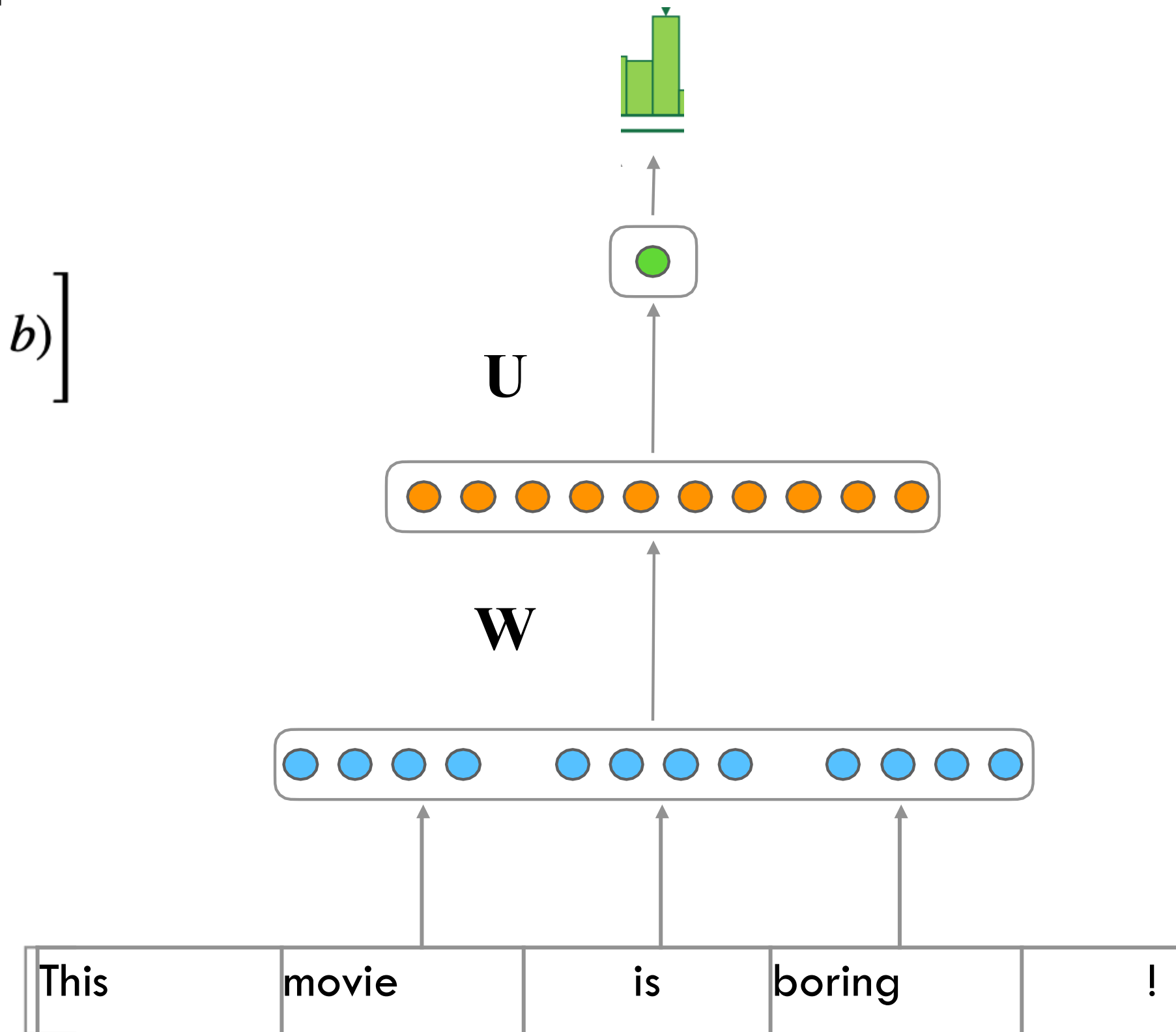
FFNN for Classification

FNN and Classification

- Training Objective: For each training example $(\mathbf{x}, y)$, our objective is to maximize the probability of the correct class y or we can minimize the negative log probability of that class:

$$L_{CE} = -\log P(y = c | \mathbf{x}; \theta) = -(\mathbf{w}_c \cdot \mathbf{x} + b) + \log \left[\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b) \right]$$

- Loss as Cross entropy: $H(p, q) = -\sum_{i=1}^C p_i \log q_i$
 - ground truth (or true or gold or target) is a 1-hot vector, $p = [0, \dots, 0, 1, 0, \dots, 0]$, then:
 - hence, the only term left is the negative log probability of the true class y_i^* : $-\log p(y_i | x_i)$
 - True for both language modeling and classification



Training FFNNs

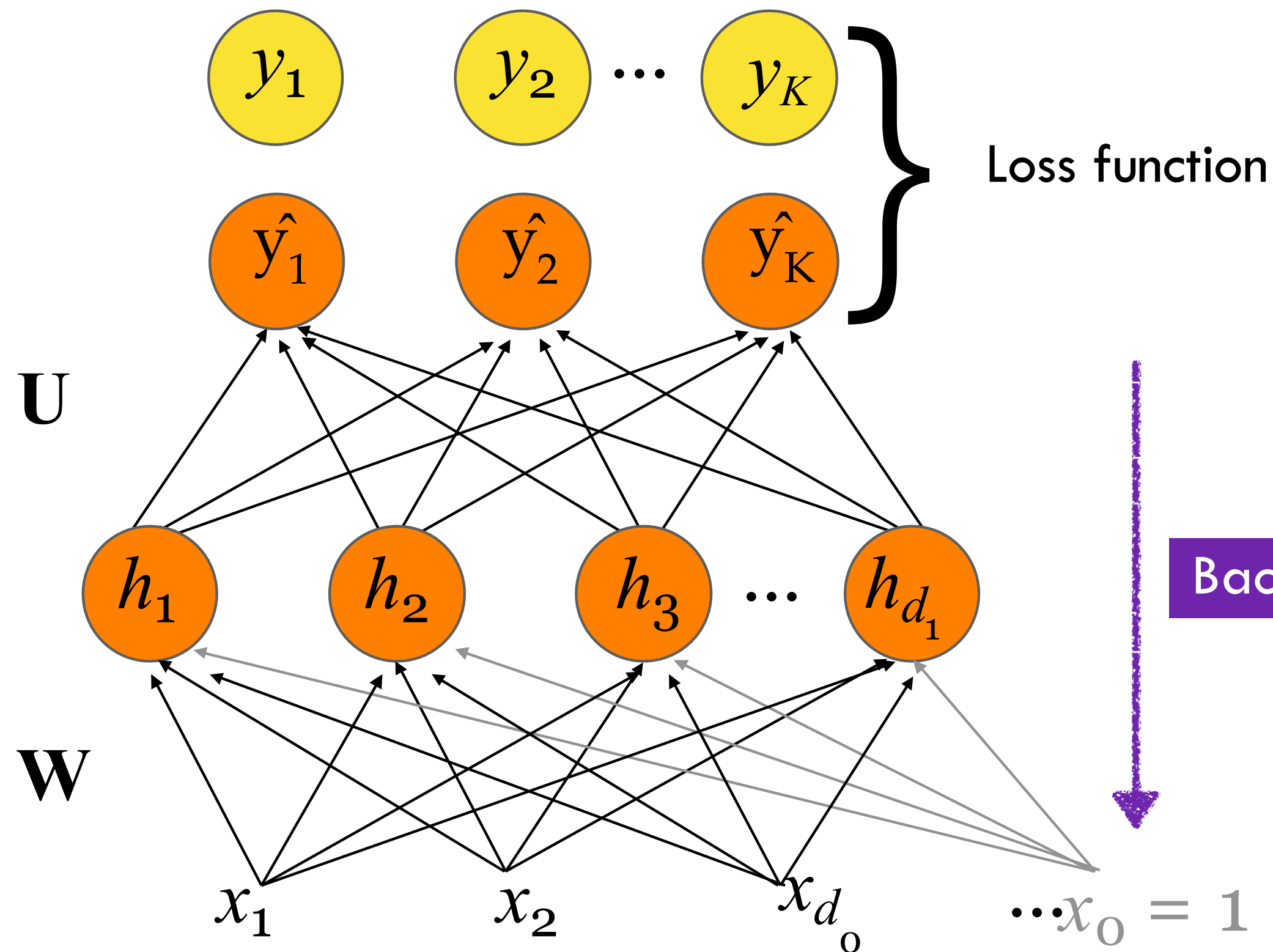
Intuition: Training a 2-layer Network

Training instance $\mathbf{y}$

Model Output $\hat{\mathbf{y}} = \text{softmax}(\mathbf{U} \cdot \mathbf{h})$

Forward Pass

Training instance



Intuition: Training a 2-layer network

For every training tuple (x, y)

- Run **forward** computation to find our estimate $\hat{y}$
- Run **backward** computation to update weights:
 - For every output node
 - Compute loss L between true y and the estimated $\hat{y}$
 - For every weight w from hidden layer to the output layer
 - Update the weight
 - For every hidden node
 - Assess how much blame it deserves for the current answer
 - For every weight w from input layer to the hidden layer
 - Update the weight

Reminder: Loss Function for Binary Logistic Regression

A measure for how far off the current answer is to the right answer

Cross entropy loss for logistic regression:

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \\ &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log(1 - \sigma(w \cdot x + b))] \end{aligned}$$

Reminder: Gradient descent for weight updates

Use the derivative of the loss function with respect to weights $\frac{d}{dw} L(f(x; w), y)$

To tell us how to adjust weights for each training item

- Move them in the opposite direction of the gradient

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$

- For logistic regression

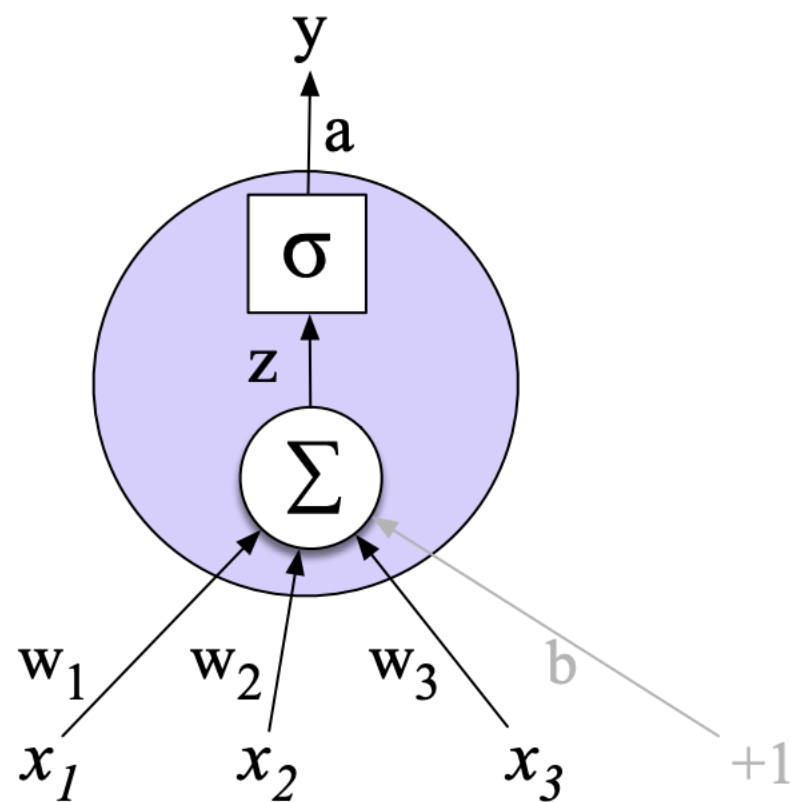
$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

Where did the derivative come from?

Use the chain rule!

$$f(x) = u(v(x))$$

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$



$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w}$$

Derivative of the weighted sum
 Derivative of the Activation
 Derivative of the Loss

LR and FFNN: Similarities and Differences

Cross Entropy Loss again!

$$\begin{aligned} L_{CE}(y, \hat{y}) &= -\log p(y|x) = -[y \log \hat{y} + (1-y) \log(1-\hat{y})] \\ &= -[y \log \sigma(\mathbf{w} \cdot \mathbf{x} + b) + (1-y) \log(\sigma(-\mathbf{w} \cdot \mathbf{x} + b))] \end{aligned}$$

Gradient Update

$$\frac{\partial L_{CE}(\hat{y}, y)}{\partial w_j} = [\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y]x_j$$

only one parameter!

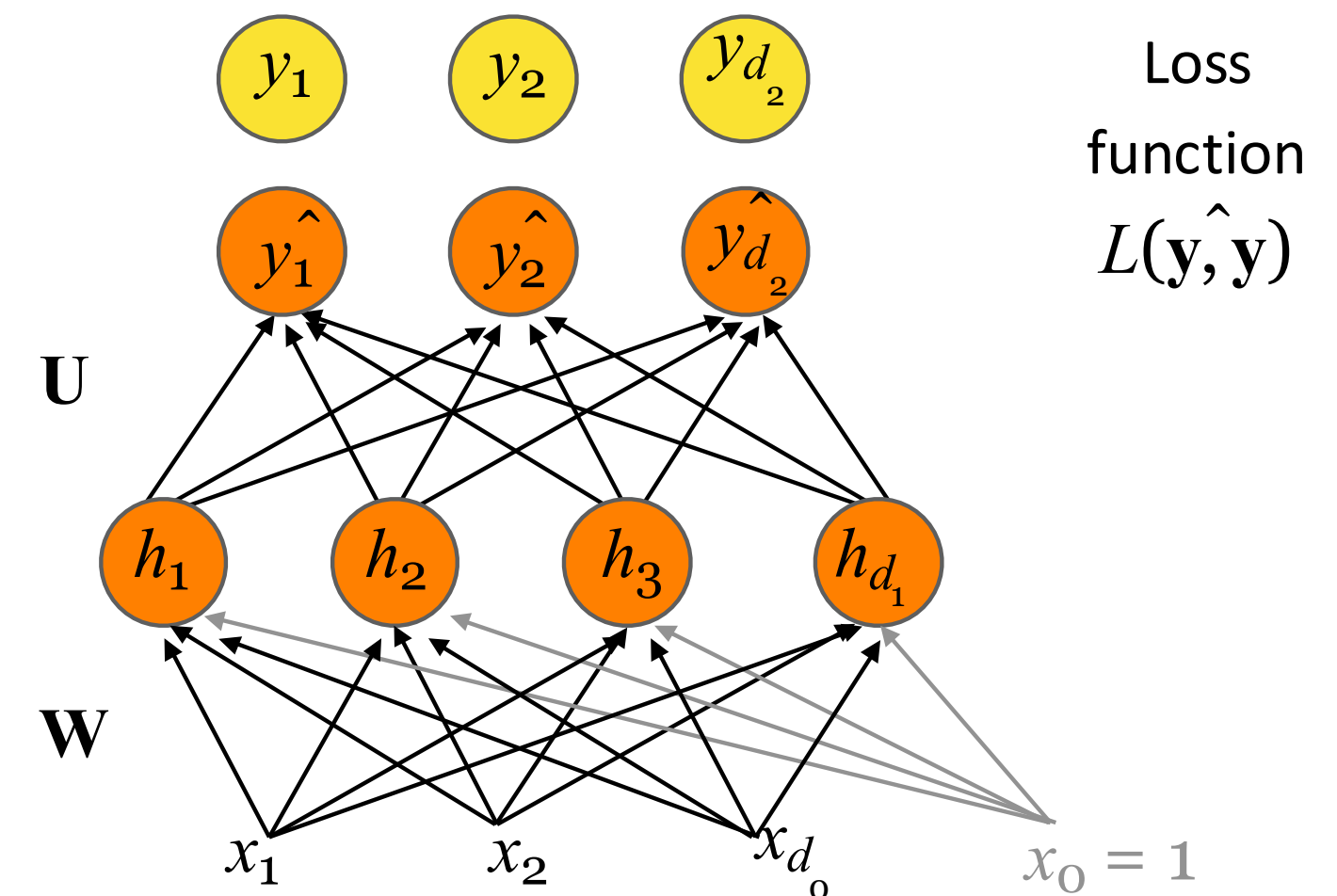
Computation Graphs

As (multiple) hidden layers are introduced, there will be many more parameters to consider, not to mention activation functions!

Computation Graphs and Backprop

Why Computation Graphs?

- For training, we need the derivative of the loss with respect to each weight in every layer of the network
 - But the loss is computed only at the very end of the network!
- Solution: error backpropagation or backward differentiation
 - Backprop is a special case of backward differentiation
 - Which relies on computation graphs



Graph representing the process of computing a mathematical expression

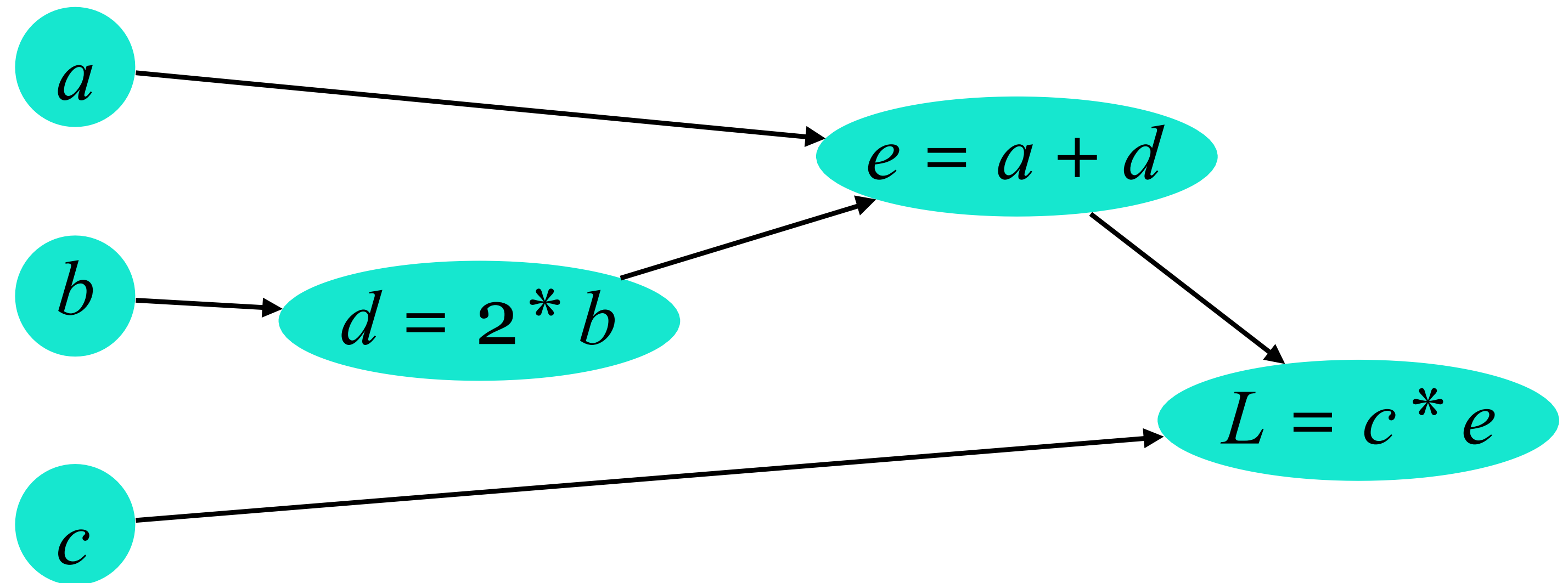
Example: Computation Graph

$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

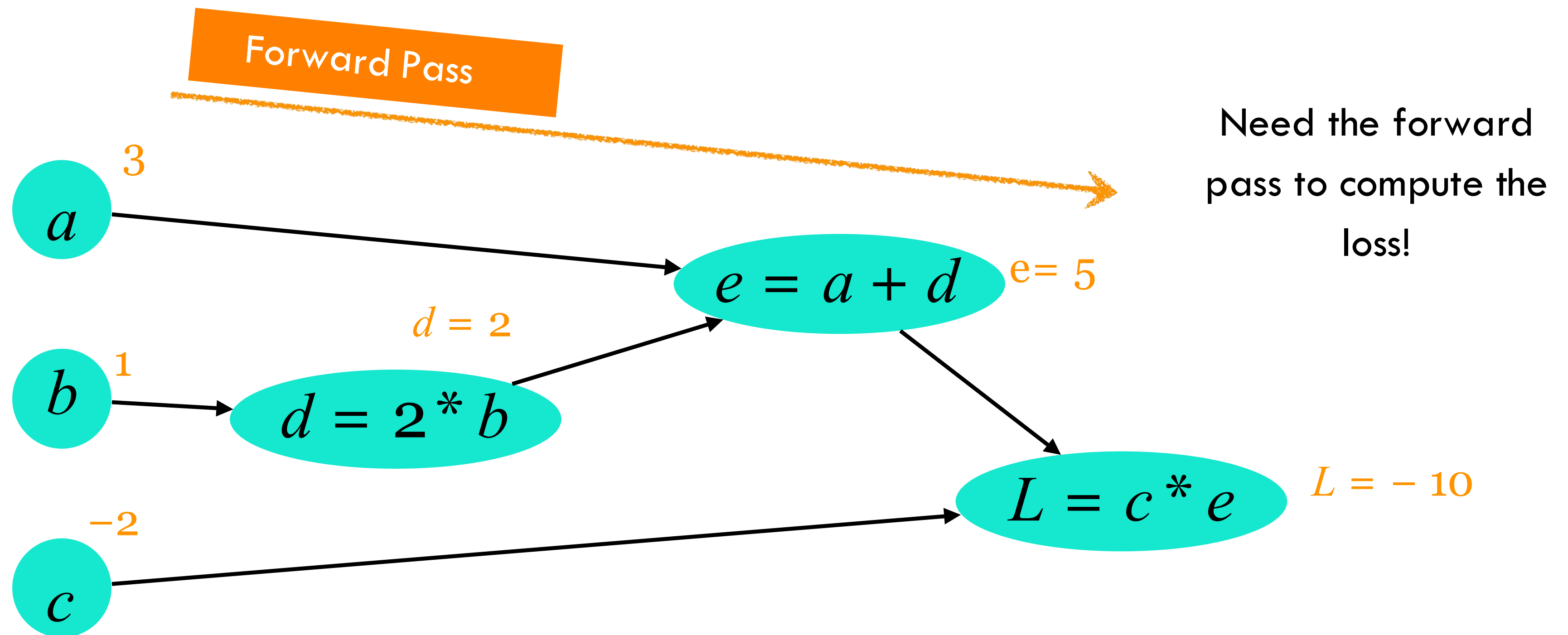
$$e = a + d$$

$$L = c * e$$



Example: Forward Pass

$$d = 2 * b$$
$$e = a + d$$
$$L = c * e$$



But how to compute parameter updates?

Example: Backward Pass Intuition

- The importance of the computation graph comes from the **backward pass**
- Used to compute the derivatives needed for the weight updates

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$

$$\left. \begin{array}{l} \frac{\partial L}{\partial a} = ? \\ \frac{\partial L}{\partial b} = ? \\ \frac{\partial L}{\partial c} = ? \end{array} \right\} \text{Input Layer Gradients}$$

$$\text{Hidden Layer Gradients} \left\{ \begin{array}{l} \frac{\partial L}{\partial d} = ? \\ \frac{\partial L}{\partial e} = ? \end{array} \right.$$

Chain Rule of Differentiation!

Example: Applying the chain rule

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$

$$\frac{\partial L}{\partial c} = e$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$

$$\frac{\partial L}{\partial e} = c$$

$$\frac{\partial L}{\partial d} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d}$$

Cannot do all at once, need to follow an order...

Example: Backward Pass

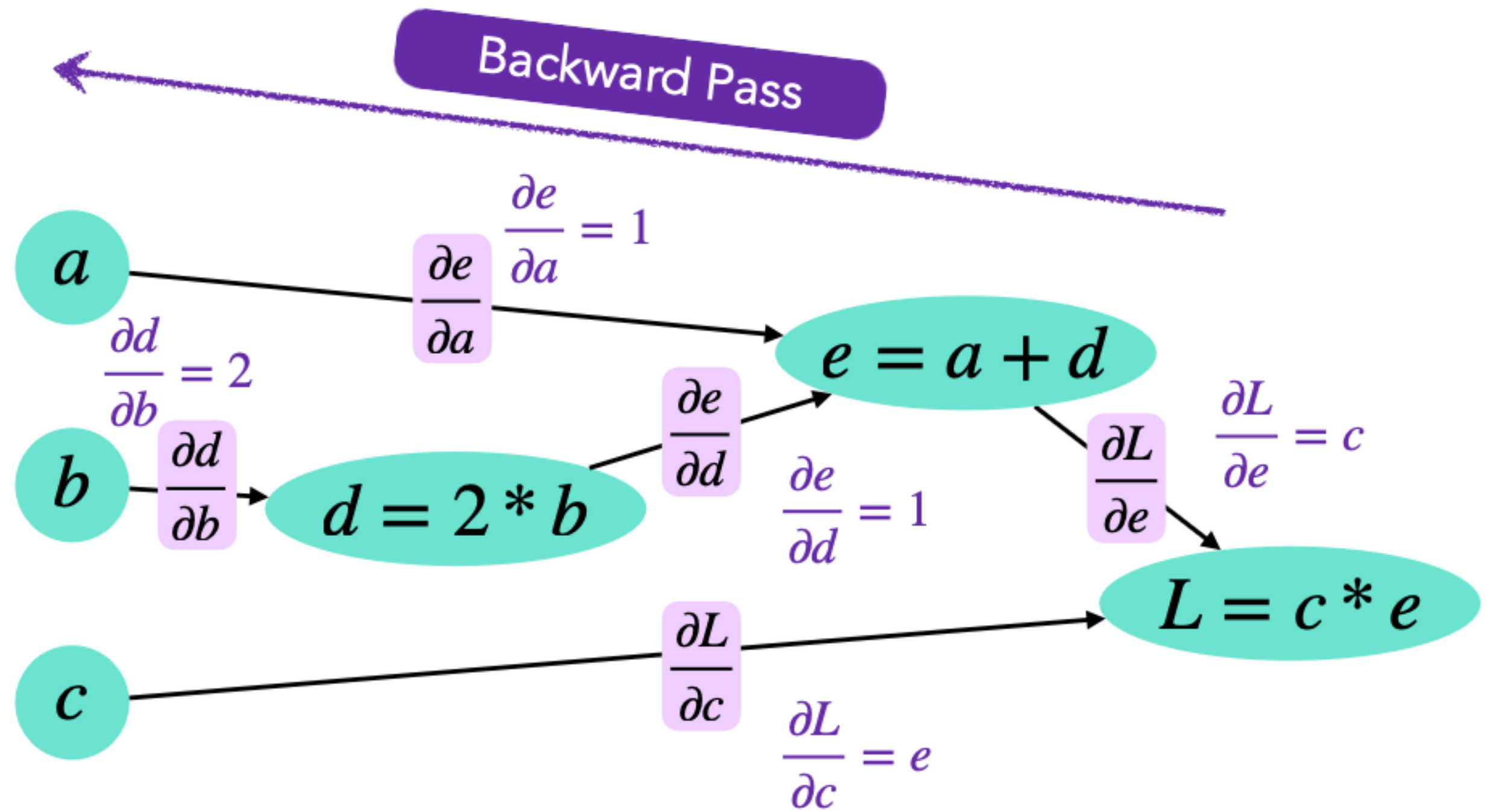
But we need the gradients of the loss with respect to parameters...

$$\frac{\partial L}{\partial c} = e \qquad \frac{\partial L}{\partial e} = c$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial d} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$



Example

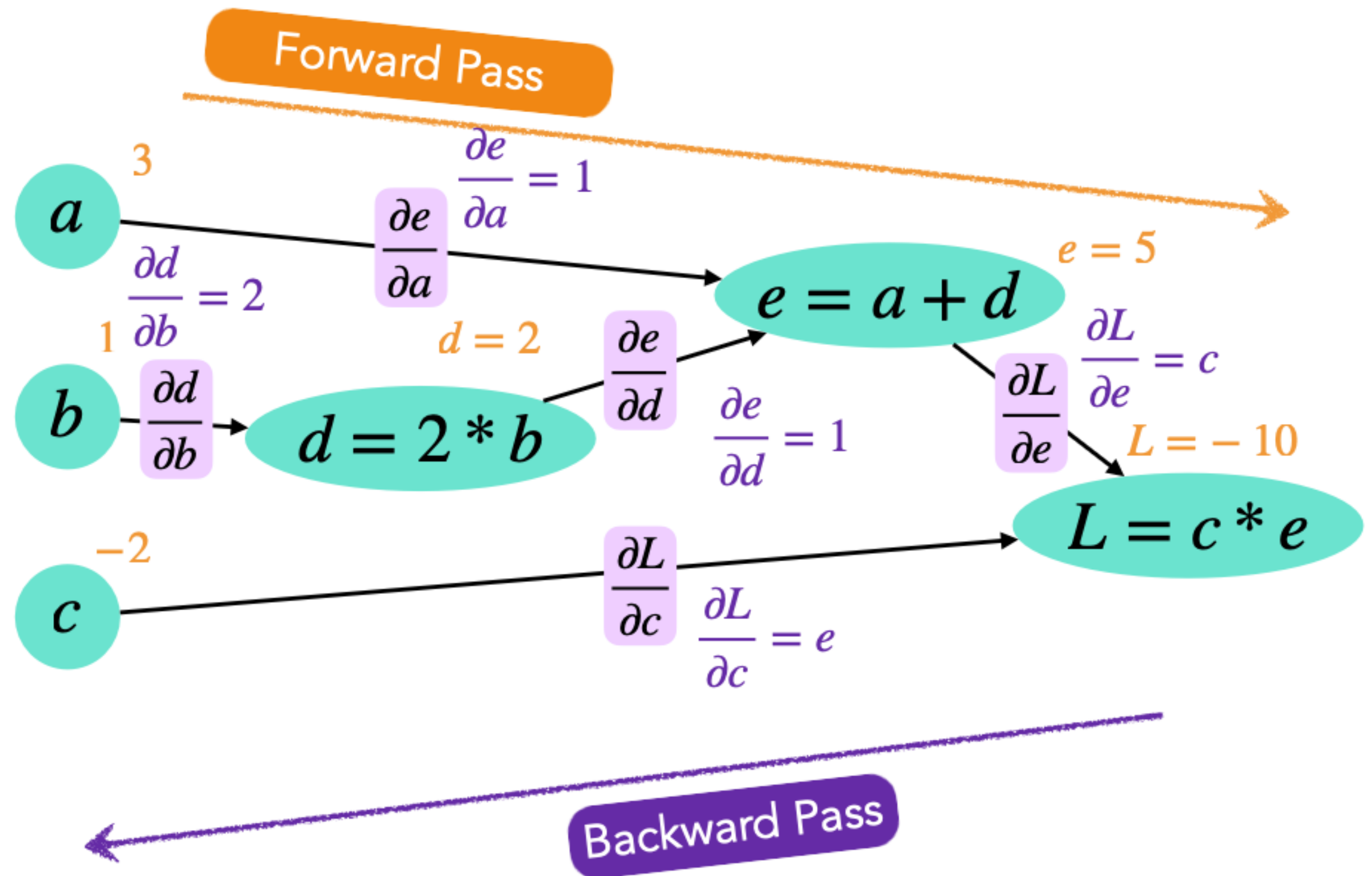
$$\frac{\partial L}{\partial e} = c = -2$$

$$\frac{\partial L}{\partial c} = e = 5$$

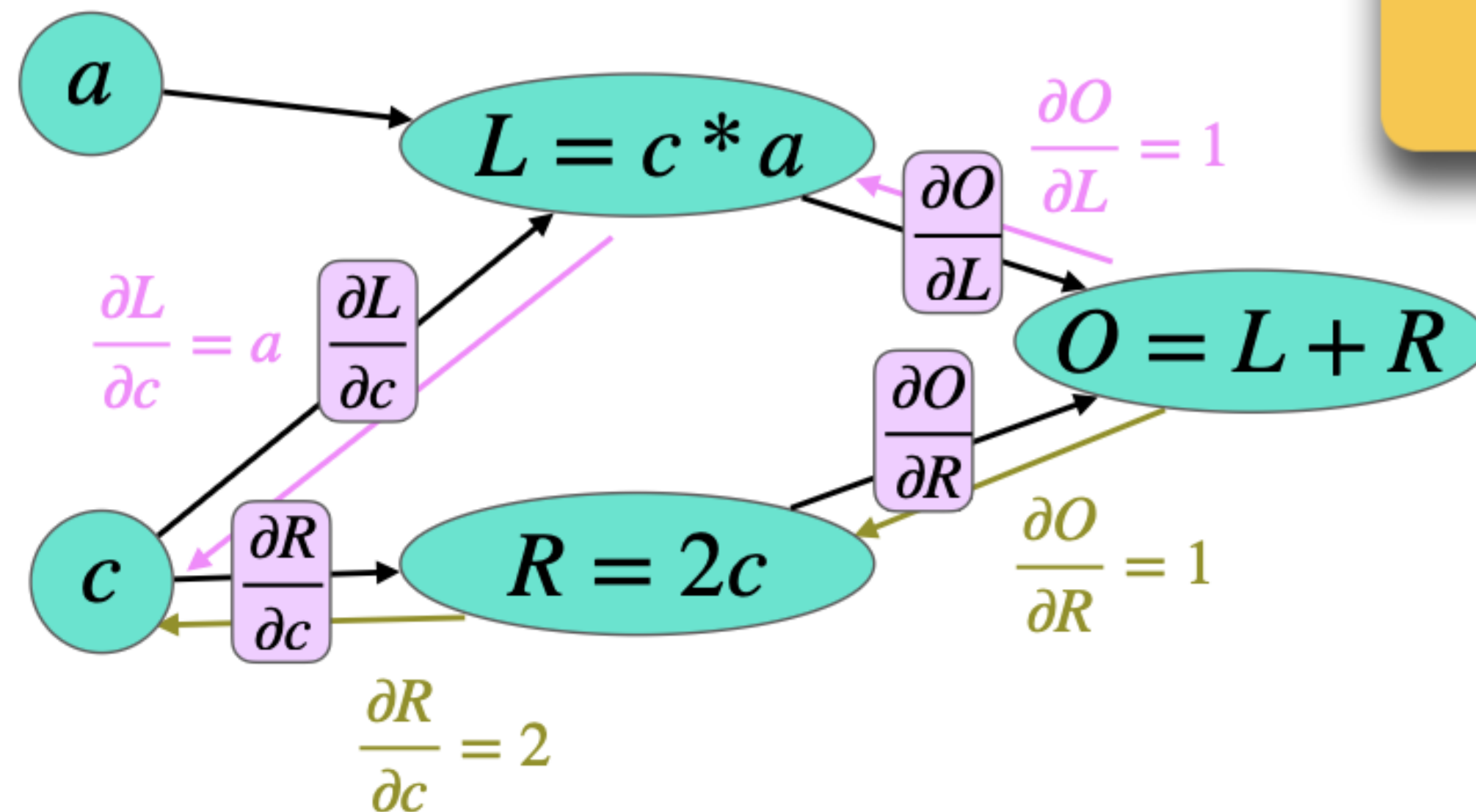
$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a} = -2$$

$$\frac{\partial L}{\partial d} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} = -2$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b} = -4$$



Example: Two Paths

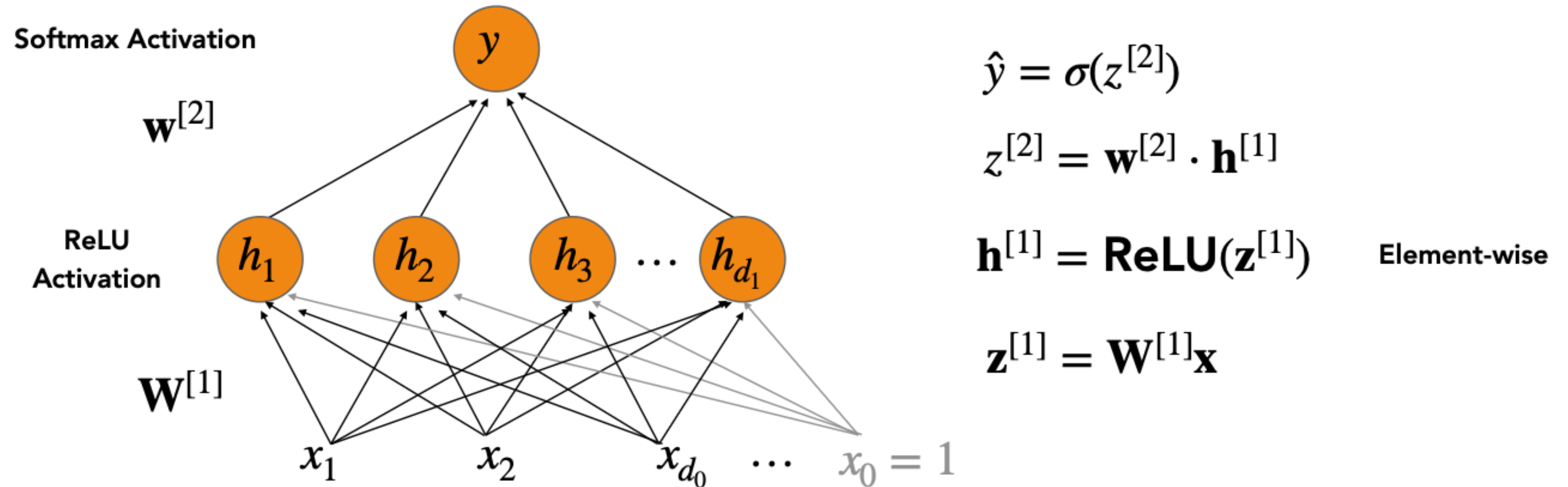


When multiple branches converge on a single node we will add these branches

$$\frac{\partial O}{\partial c} = \frac{\partial O}{\partial L} \frac{\partial L}{\partial c} + \frac{\partial O}{\partial R} \frac{\partial R}{\partial c}$$

Such cases arise when considering regularized loss functions

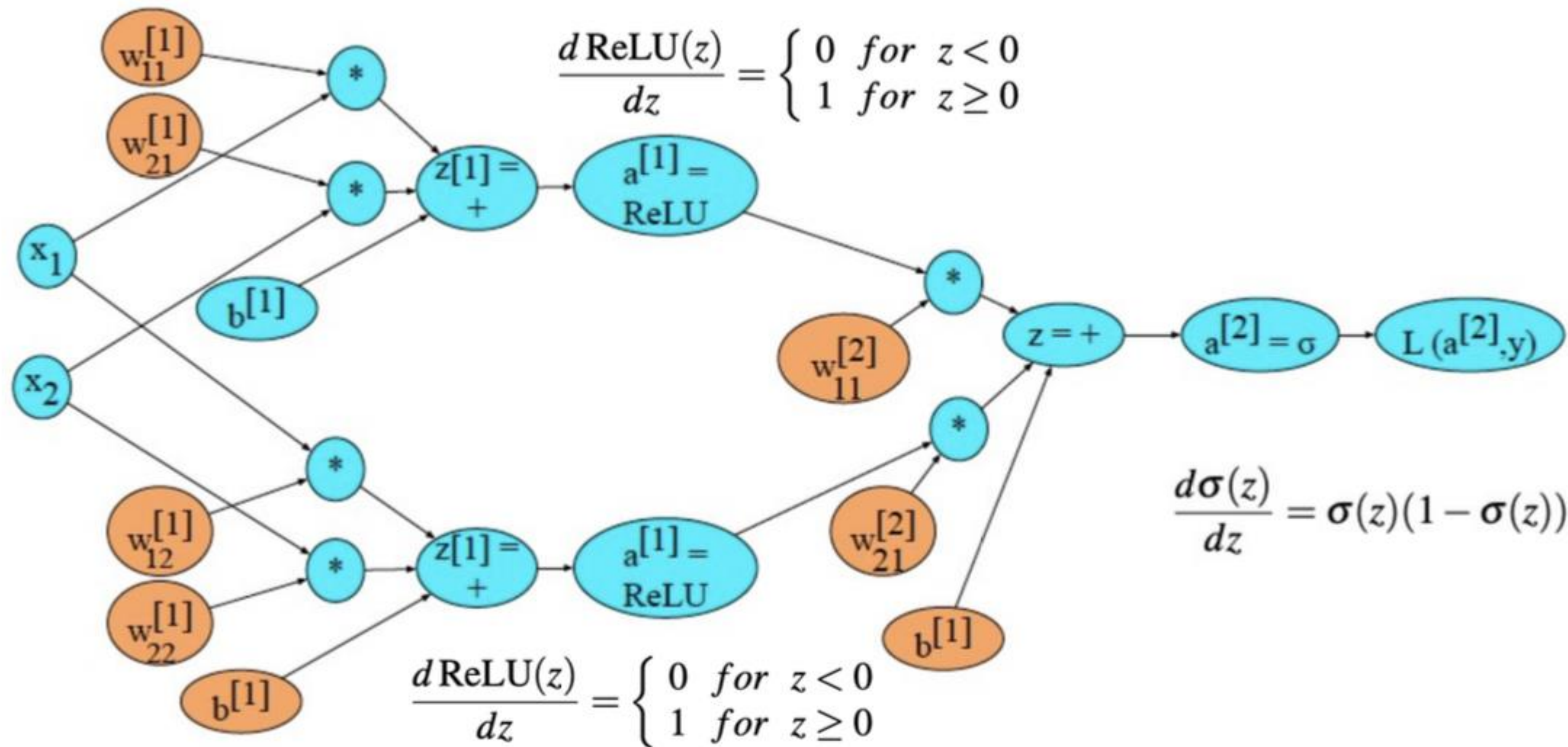
Backward Differentiation on a 2-layer MLP



$$\frac{\partial \sigma(z)}{\partial z} = \sigma(z)\sigma(-z) = \sigma(z)(1 - \sigma(z))$$

$$\frac{d \text{ReLU}(z)}{dz} = \begin{cases} 0 & \text{for } z < 0 \\ 1 & \text{for } z \geq 0 \end{cases}$$

2 layer MLP with 2 input features



Starting off the backward pass: $\frac{\partial L}{\partial \mathbf{z}}$
 (I'll write a for $a^{[2]}$ and z for $z^{[2]}$)

$$\begin{aligned} z^{[1]} &= W^{[1]} \mathbf{x} + b^{[1]} \\ a^{[1]} &= \text{ReLU}(z^{[1]}) \\ z^{[2]} &= W^{[2]} a^{[1]} + b^{[2]} \\ a^{[2]} &= \sigma(z^{[2]}) \\ \hat{y} &= a^{[2]} \end{aligned}$$

$$L(\hat{y}, y) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

$$L(a, y) = -(y \log a + (1 - y) \log(1 - a))$$

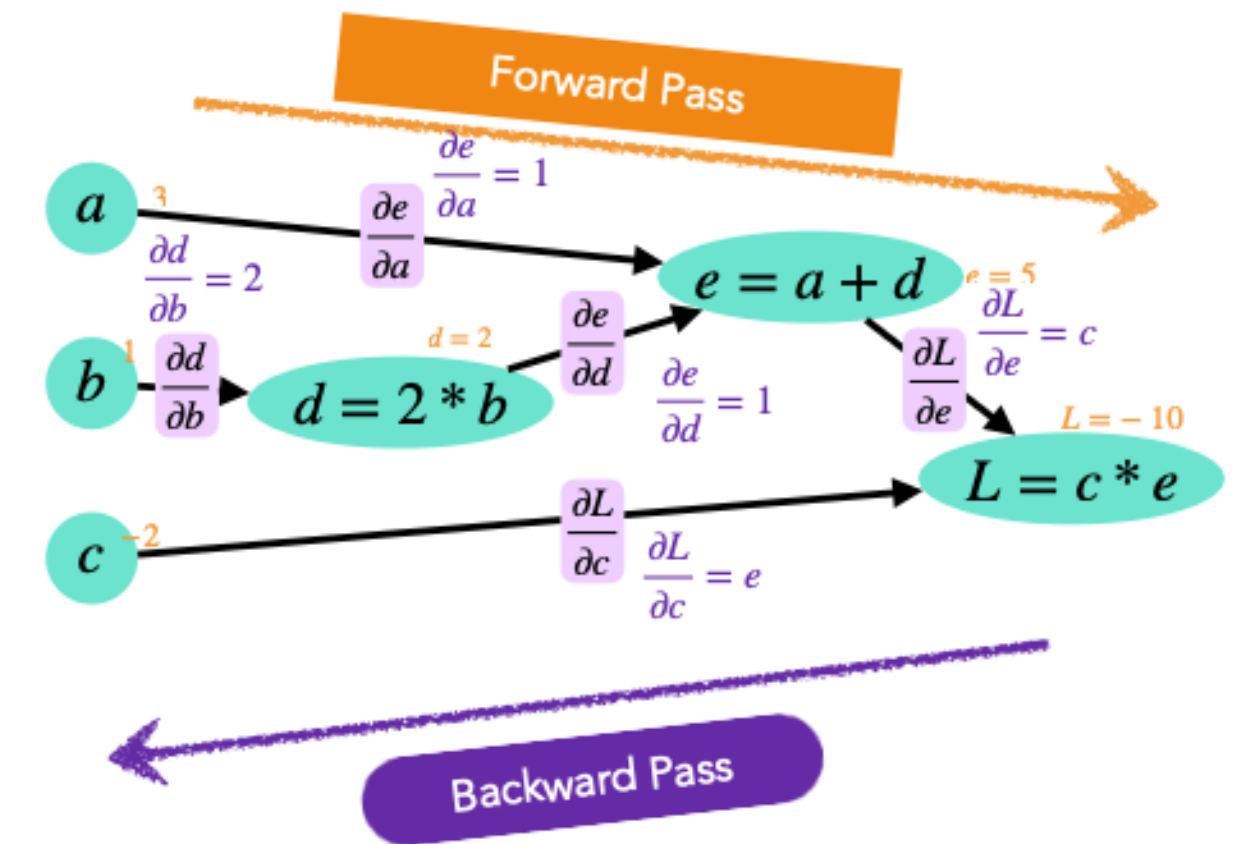
$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z}$$

$$\begin{aligned} \frac{\partial L}{\partial a} &= - \left(\left(y \frac{\partial \log(a)}{\partial a} \right) + (1 - y) \frac{\partial \log(1 - a)}{\partial a} \right) \\ &= - \left(\left(y \frac{1}{a} \right) + (1 - y) \frac{1}{1 - a} (-1) \right) = - \left(\frac{y}{a} + \frac{y - 1}{1 - a} \right) \end{aligned}$$

$$\frac{\partial a}{\partial z} = a(1 - a) \quad \frac{\partial L}{\partial z} = - \left(\frac{y}{a} + \frac{y - 1}{1 - a} \right) a(1 - a) = a - y$$

Summary: Backprop / Backward Differentiation

- For training, we need the derivative of the loss with respect to weights in early layers of the network
 - But loss is computed only at the very end of the network!
- Solution: backward differentiation



Given a computation graph and the derivatives of all the functions in it we can automatically compute the derivative of the loss with respect to these early weights.

Libraries such as PyTorch do this for you in a single line: `model.backward()`